

Memoria Técnica Ecosistemas para Prácticas Formativas: FTC Gestión + Odoo



Autor: David Romero

Ciclo GS: 2º Desarrollo Aplicaciones Multiplataforma

Tutora: Maria Sierra Escalera

Curso: 2026

Módulo: Proyecto Intermodular

Índice de Tablas.....	4
Índice de Figuras.....	4
1. Introducción.....	5
1.2. Objetivo.....	5
Objetivo general.....	5
Objetivos específicos.....	5
1.3. Alcance y limitaciones.....	6
Alcance.....	6
Limitaciones.....	6
2. Análisis.....	6
2.1 Requisitos Funcionales.....	7
2.2 Requisitos No Funcionales.....	8
2.2 Casos de Uso.....	9
CU01 – Autenticación de usuario.....	9
CU02 – Gestión de entidades del sistema.....	10
CU03 – Asignación de alumnos a tutores y empresas.....	11
CU04 – Creación de tareas.....	11
CU05 – Gestión del estado de tareas.....	12
CU06 – Validación de tareas.....	13
CU07 – Registro de asistencia.....	13
CU08 – Seguimiento de asistencia y tareas.....	14
CU09 – Sincronización externa.....	15
CU10 – Generación de informes PDF.....	15
CU11 – Predicción de horas.....	16
2.3 Actores del sistema.....	16
Profesor gestor.....	17
Tutor de empresa.....	17
Alumno.....	17
Profesor Tutor.....	17
Sistema externo (ERP / Odoo).....	17
Microservicio de predicción (IA).....	17
2.4 Diagrama de Casos de Uso.....	18
Entidades del sistema principal.....	20
Sistema externo (ERP).....	22
Servicio externo de predicción.....	23
3. Diseño.....	23
3.1 Arquitectura del Sistema.....	23
Diagrama Arquitectura de Sistemas.....	23
Sistema principal (Backend - Spring Boot).....	23
Frontend.....	24
Sistema externo ERP (Odoo).....	24
Microservicio de predicción (FastAPI).....	24
Bases de datos.....	24
Despliegue y orquestación.....	24

3.2 Diagrama de Clases (UML).....	25
3.3 Diagramas de Secuencia.....	26
3.4 Diagramas de Estado.....	27
4. Tecnologías y Entornos de desarrollo.....	28
4.1 Stack tecnológico.....	28
Diagrama Entidad-Relación.....	30
Diccionario de Datos (Selección de tablas principales).....	30
Otras entidades.....	32
5. Implementación.....	32
5.1 Configuración del Entorno y Orquestación (Docker).....	32
Configuración del Dockerfile en el Frontend.....	32
Red Global y Resolución de Nombres.....	32
Persistencia y Gestión de Volúmenes.....	33
Resolución de Conflictos y Desafíos Técnicos.....	33
Resumen de Servicios Orquestados.....	33
5.2 Desarrollo del Backend (Spring Boot).....	34
Arquitectura y Estructura de Paquetes.....	34
Interfaz de Programación de Aplicaciones (API REST).....	34
Estandarización de respuestas.....	35
Documentación y Pruebas.....	35
Transferencia de Datos y Mapeo (DTO y Mappers).....	35
Comunicación con el Microservicio de IA.....	35
Lógica de Asistencias y Gestión de Datos.....	36
Desafíos Técnicos y Resolución de Errores (CORS).....	36
Autenticación y Gestión de Roles.....	36
5.3 El Microservicio de Predicción (FastAPI + ML).....	37
Creación del modelo de predicción.....	37
El truco de los selectores (Ingeniería de datos).....	38
Funcionamiento técnico.....	38
5.4 Integración con Odoo (Módulo Custom).....	39
Estructura de Módulos y Organización.....	39
El Asistente de Sincronización (Wizard).....	40
Generación de Informes PDF.....	41
Notas sobre Seguridad y Código.....	42
5.5 Frontend y Experiencia de Usuario (React).....	43
Arquitectura de Aplicación de Página Única (SPA).....	43
Gestión de Estado Global con Context API.....	43
Control de Acceso y Dashboards Independientes.....	43
Navegación Dinámica y Adaptabilidad (Responsividad).....	44
Componentes Reutilizables y Eficiencia.....	45
Flujo Operativo en la Vista de Alumno.....	45
Gestión de Visualización: Paginación en Cliente (AppTable).....	45
Detalles de la implementación técnica.....	45
6. Despliegue y Pruebas.....	45

Pruebas de Orquestación Final.....	46
6.1 Puesta en marcha del sistema y despliegue.....	46
Procedimiento de despliegue con Docker Compose.....	46
Configuración inicial e instalación de módulos en el ERP.....	47
6.2 Pruebas de integración y Validación APIs.....	48
Validación del Backend con Postman.....	48
Pruebas de Microservicio de IA con Swagger.....	50
Resultados de las pruebas de integración.....	50
6.3 Validación de la Interfaz y Flujos de Usuario (Frontend).....	51
Control de Acceso y Seguridad de Rutas.....	51
Ciclo de Vida de las Entidades y Roles.....	52
7. Conclusiones y Valoración.....	55
7.1 Consecución de objetivos.....	55
Objetivos Específicos Alcanzados.....	55
Superación de Expectativas: El Módulo de IA.....	55
Valoración de la Coordinación.....	55
7.2 Desafíos Transversales y Gestión de Riesgos.....	56
7.3 Análisis de coste.....	56
Costes de Personal (Recursos Humanos).....	56
Costes de Software e Infraestructura.....	57
Resumen de Inversión Total.....	57
7.4 Líneas de trabajo futuro.....	57
Interfaz y Experiencia de Usuario (UX/UI).....	58
Seguridad y Arquitectura.....	58
Funcionalidades de Gestión y Comunicación.....	58
Escalabilidad e Inteligencia Artificial.....	58
Despliegue y Calidad.....	58
7.5 Conclusión final.....	59
Relación con los Módulos del Ciclo.....	59
Reflexión Personal y Aprendizaje.....	59
8. Bibliografía.....	59
8.1 Documentación oficial de tecnologías.....	60
8.2 Documentación teórica asignaturas.....	60
8.3. Herramientas de Asistencia y Productividad.....	60

Índice de Tablas

Tabla Requisitos funcionales.....	7
Tabla Recintos No Funcionales.....	8
Tabla Casos de Uso.....	9
Actores del Sistema.....	16
Historia de usuario.....	19
Tabla: Asistencias.....	31
Tabla: Tareas.....	31
Tabla Endpoints Spring.....	34
Tabla Coste Personal.....	56
Tabla Costes Software.....	57

Índice de Figuras

Figura 1. Diagrama de Casos de uso.....	18
Figura 2. Diagrama de Arquitectura de Sistemas.....	23
Figura 3. Diagrama de Clases.....	25
Figura 4. Diagrama de Secuencia Asistencia.....	26
Figura 5. Diagrama de Secuencia Predicción Horas.....	26
Figura 6. Diagrama de Secuencia Sincronización con Spring.....	27
Figura 7. Diagrama de Estado Tarea.....	27
Figura 8. Diagrama entidad relación bbdd (MySQL) FTC Gestión.....	30
Figura 10. Gráfico de Dispersión de Modelo RandomForest.....	38
Figura 11. Swagger de fastAPI.....	39
Figura 13. Asistente para la sincronización de datos con Spring.....	41
Figura 15. PDF generado para el alumno.....	42
Figura 19. Vista de terminal una vez ejecutado el docker-compose up.....	47
Figura 20. Pantalla de carga módulos desarrollados.....	48
Figura 21. Método post aplicado al login en Postman.....	49
Figura 22. Método Post aplicado a la predicción de horas en Swagger.....	50
Figura 23. Error de login.....	51
Figura 24. Modal para añadir un alumno al sistema.....	52
Figura 25. Tabla con el alumno guardado.....	53
Figura 26. Dashboard de Asistencias para Alumno.....	54

1. Introducción

Durante la formación en un ciclo formativo de grado superior, una de las fases más relevantes es la realización de prácticas en empresa, donde el alumnado aplica los conocimientos adquiridos en un entorno profesional real.

En este contexto, resulta fundamental llevar un control adecuado de la asistencia, así como de las tareas desarrolladas, además de mantener una comunicación fluida entre el alumno, el tutor docente y el tutor laboral.

Sin embargo, en muchos casos este seguimiento se realiza de forma manual o mediante herramientas dispersas, lo que dificulta la organización y el control del progreso del alumno.

Por ello, surge la necesidad de desarrollar una aplicación que centralice la gestión de la asistencia, el registro de actividades y la comunicación entre los distintos actores implicados.

1.2. Objetivo

Objetivo general

Desarrollar una aplicación que permita gestionar y centralizar el seguimiento de las prácticas en empresa de alumnos de formación profesional.

Objetivos específicos

- Proporcionar al gestor de la aplicación la posibilidad de gestionar al resto de usuarios y entidades del sistema.
- Registrar la asistencia diaria del alumno.
- Facilitar la asignación de tareas al alumnado durante el periodo de prácticas.
- Permitir el seguimiento del progreso del alumno en función de las tareas realizadas.
- Proporcionar un mecanismo de validación de las tareas por parte del tutor docente.
- Mejorar la coordinación entre alumno, tutor laboral y tutor docente.
- Centralizar la información relacionada con el desarrollo de las prácticas en una única plataforma.

1.3. Alcance y limitaciones

Alcance

La aplicación desarrollada cubre la gestión del seguimiento de las prácticas en empresa del alumnado de formación profesional. En concreto, el sistema permite:

- La gestión de usuarios del sistema (administrador, alumno, tutor docente y tutor laboral).
- El registro de asistencia del alumnado durante el periodo de prácticas.
- La creación y asignación de tareas por parte del tutor laboral.
- El seguimiento del estado de las tareas por parte del alumno.
- La validación de tareas por parte del tutor docente.
- La visualización del progreso del alumno a través del estado de sus tareas.
- La centralización de toda la información relacionada con el seguimiento de las prácticas en una única plataforma.
- Fuente de datos generados para ERP externo como repositorio espejo.

Limitaciones

El sistema no contempla determinadas funcionalidades que quedan fuera del alcance del proyecto actual, entre las que se incluyen:

- Gestión de convenios de prácticas con empresas o entidades educativas.
- Sistema de mensajería directa entre los usuarios de la aplicación.
- Gestión administrativa completa del centro educativo.

2. Análisis

En este apartado se definen los requisitos del sistema desarrollado, así como los distintos actores que interactúan con la aplicación y las funcionalidades principales que esta debe ofrecer. El objetivo es concretar de forma estructurada el comportamiento esperado del sistema antes de su diseño e implementación.

2.1 Requisitos Funcionales

Tabla Requisitos funcionales		
ID	Título	Requisito
RF01	<i>Login por roles</i>	El sistema permitirá autenticación mediante email y contraseña con selección de rol.
RF02	<i>Gestión de entidades</i>	El sistema permitirá al profesor gestor crear centros educativos, empresas y usuarios.
RF03	<i>Asignación de usuarios</i>	El sistema permitirá al profesor gestor asignar alumnos a tutores docentes y empresas.
RF04	<i>Creación de tareas</i>	El sistema permitirá al tutor de empresa crear tareas.
RF05	<i>Asignación de trabajo</i>	El sistema permitirá asignar tareas a alumnos.
RF06	<i>Control de asistencia</i>	El sistema permitirá al alumno registrar asistencia diaria.
RF07	<i>Estado de tareas</i>	El sistema permitirá al alumno actualizar el estado de las tareas.
RF08	<i>Supervisión</i>	El sistema permitirá al tutor docente visualizar alumnos, asistencia y tareas.
RF09	<i>Validación</i>	El sistema permitirá al tutor docente validar tareas completadas.
RF10	<i>Sincronización con ERP</i>	El sistema permitirá la sincronización de datos con un ERP como repositorio espejo externo.
RF11	<i>Generación de informes</i>	El sistema permitirá la generación de informes en formato PDF desde el sistema externo.
RF12	<i>Predicción de horas</i>	El sistema integrará un microservicio de predicción para estimación de horas de tareas.

2.2 Requisitos No Funcionales

Tabla Recintos No Funcionales		
ID	Título	Requisito
RNF01	<i>Seguridad de acceso</i>	El sistema deberá garantizar la autenticación de usuarios mediante email y contraseña, verificando las credenciales contra la base de datos del sistema.
RNF02	<i>Control de autorización</i>	El sistema deberá restringir el acceso a las funcionalidades en función del rol del usuario autenticado.
RNF03	<i>Diseño Responsive</i>	El sistema deberá ofrecer una interfaz web adaptativa, utilizando tecnologías adecuadas para su correcta visualización en dispositivos de distintos tamaños.
RNF04	<i>Compatibilidad</i>	El sistema deberá ser accesible desde los principales navegadores web, siendo compatible principalmente con Firefox y Chrome en modo privado.
RNF05	Escalabilidad y Carga	El sistema deberá garantizar un rendimiento adecuado para un entorno de uso limitado, estimado en un máximo de aproximadamente 100 usuarios concurrentes.
RNF06	<i>Integridad de datos</i>	El sistema deberá asegurar la integridad de los datos almacenados en una base de datos relacional, manteniendo la consistencia entre entidades relacionadas.
RNF07	<i>Portabilidad y Despliegue</i>	El sistema deberá ser fácilmente desplegable en diferentes entornos mediante el uso de tecnologías de contenedorización, permitiendo la orquestación de todos los servicios necesarios de forma centralizada.
RNF08	<i>Interoperabilidad</i>	El sistema deberá garantizar la interoperabilidad con sistemas externos de gestión empresarial mediante intercambio controlado de datos.

2.2 Casos de Uso

Tabla Casos de Uso		
ID	Caso de uso	Actor principal
CU01	Autenticación de usuario	Todos los roles
CU02	Gestión de entidades del sistema	Profesor gestor
CU03	Asignación de alumnos a tutores y empresas	Profesor gestor
CU04	Creación y gestión de tareas	Tutor de empresa
CU05	Gestión del estado de tareas	Alumno
CU06	Validación de tareas	Tutor docente
CU07	Registro de asistencia	Alumno
CU08	Seguimiento de asistencia y tareas	Tutor docente
CU09	Sincronización con sistema externo	Sistema (Odoos)
CU10	Generación de informes PDF	Sistema externo
CU11	Predicción de horas de tareas	Sistema externo (microservicio IA)

CU01 – Autenticación de usuario

Actor principal: Todos los roles

Descripción:

El sistema permite el acceso de los usuarios mediante la introducción de email, contraseña y la selección del rol correspondiente. A partir de esta autenticación, el sistema determina qué funcionalidades están disponibles para cada usuario según su perfil (alumno, tutor docente, tutor de empresa o profesor gestor).

Flujo principal:

1. El usuario accede a la pantalla de inicio de sesión.
2. Introduce su email, contraseña y selecciona su rol.
3. El sistema valida las credenciales contra la base de datos.
4. Si los datos son correctos, el sistema identifica el rol del usuario.
5. El sistema redirige al usuario a su área correspondiente dentro de la aplicación.

Flujos alternativos:

- Si las credenciales son incorrectas, el sistema muestra un mensaje de error y no permite el acceso.
- Si el rol seleccionado no coincide con el asociado al usuario, el acceso es denegado.

Resultado:

El usuario accede al sistema con su perfil correctamente identificado, pudiendo utilizar únicamente las funcionalidades asociadas a su rol.

CU02 – Gestión de entidades del sistema

Actor principal: Profesor gestor

Descripción:

El profesor gestor es el encargado de crear y administrar las entidades base del sistema, como centros educativos, empresas y usuarios (alumnos, tutores docentes y tutores de empresa). Esta funcionalidad permite configurar la estructura organizativa inicial necesaria para el funcionamiento de la aplicación.

Flujo principal:

1. El profesor gestor accede al módulo de administración del sistema.
2. El sistema muestra las opciones de gestión de entidades (centros, empresas y usuarios).
3. El profesor gestor selecciona el tipo de entidad que desea crear o gestionar.
4. El sistema muestra el formulario correspondiente.
5. El profesor gestor introduce los datos de la nueva entidad o modifica una existente.
6. El sistema valida la información introducida.
7. El sistema guarda los cambios en la base de datos.
8. El sistema confirma la operación realizada.

Flujos alternativos:

- Si los datos introducidos no son válidos, el sistema muestra un mensaje de error y solicita corrección.
- Si la entidad ya existe (por ejemplo, email o identificador duplicado), el sistema impide su creación.

Resultado:

Las entidades del sistema quedan correctamente registradas y disponibles para su uso en el resto de funcionalidades.

CU03 – Asignación de alumnos a tutores y empresas

Actor principal: Profesor gestor

Descripción:

El profesor gestor establece las relaciones entre alumnos, tutores docentes y tutores de empresa, asignando cada alumno tanto a su tutor correspondiente como a la empresa donde realizará las prácticas. Esta funcionalidad permite organizar la estructura de seguimiento del alumnado durante el periodo de prácticas.

Flujo principal:

1. El profesor gestor accede al módulo de asignación de alumnos.
2. El sistema muestra la lista de alumnos disponibles sin asignar o con asignación modificable.
3. El profesor gestor selecciona un alumno.
4. El sistema muestra los tutores docentes y empresas disponibles.
5. El profesor gestor selecciona un tutor docente y una empresa para el alumno.
6. El sistema valida la coherencia de la asignación.
7. El sistema guarda las relaciones establecidas en la base de datos.
8. El sistema confirma la asignación realizada.

Flujos alternativos:

- Si el alumno ya está asignado, el sistema permite modificar la asignación previa.
- Si no existen tutores o empresas disponibles, el sistema impide completar la asignación.
- Si los datos no son válidos, el sistema muestra un mensaje de error.

Resultado:

El alumno queda correctamente asignado a un tutor docente y a una empresa, permitiendo su seguimiento dentro del sistema.

CU04 – Creación de tareas

Actor principal: Tutor de empresa

Descripción:

El tutor de empresa crea una tarea y la asigna directamente a uno o varios alumnos. La tarea se crea en estado ASIGNADA. Durante este proceso, el sistema solicita una estimación de horas a un servicio externo de predicción para complementar la información de la tarea.

Flujo principal:

1. El tutor de empresa accede al módulo de tareas.
2. El sistema muestra la opción de creación de nueva tarea.
3. El tutor de empresa introduce los datos de la tarea (título, descripción, fecha límite).
4. El tutor de empresa selecciona uno al que asignar la tarea.
5. El sistema valida la información introducida.

6. El sistema solicita al servicio externo la predicción de horas estimadas para la tarea.
7. El sistema recibe la estimación de horas y la asocia a la tarea.
8. El sistema crea la tarea en estado ASIGNADA.
9. El sistema guarda la tarea en la base de datos.
10. El sistema confirma la creación de la tarea.

Flujos alternativos:

- Si los datos introducidos no son válidos, el sistema muestra un mensaje de error y no crea la tarea.
- Si no se selecciona ningún alumno, el sistema impide la creación de la tarea.
- Si el servicio externo de predicción no está disponible, el sistema crea la tarea por defecto con estimación de 0 horas.

Resultado:

La tarea queda creada correctamente, asignada a los alumnos seleccionados y registrada en estado ASIGNADA, con o sin estimación de horas según la disponibilidad del servicio externo.

CU05 – Gestión del estado de tareas

Actor principal: Alumno

Descripción:

El alumno gestiona el estado de las tareas que tiene asignadas durante el desarrollo de sus prácticas. Puede avanzar en el flujo de la tarea según su progreso o solicitar ayuda, lo que implica la devolución de la tarea a un estado anterior.

Flujo principal:

1. El alumno accede al listado de tareas asignadas.
2. El sistema muestra las tareas junto con su estado actual.
3. El alumno selecciona una tarea.
4. El sistema muestra las acciones disponibles según el estado de la tarea.
5. El alumno cambia el estado de la tarea a EN_PROGRESO o COMPLETADA según su avance.
6. El sistema actualiza el estado de la tarea en la base de datos.
7. El sistema confirma la actualización.

Flujos alternativos:

- Si el alumno solicita ayuda, el sistema cambia el estado de la tarea a ASIGNADA.
- Si la tarea ya está COMPLETADA, el sistema no permite cambios de estado.
- Si el estado seleccionado no es válido, el sistema muestra un mensaje de error.

Resultado:

El estado de la tarea queda actualizado correctamente reflejando el progreso del alumno o la solicitud de ayuda.

CU06 – Validación de tareas**Actor principal:** Tutor docente**Descripción:**

El tutor docente revisa las tareas que han sido completadas por los alumnos y procede a su validación. Esta validación confirma que la tarea se ha realizado correctamente y cumple los requisitos establecidos, pasando al estado VALIDADA.

Flujo principal:

1. El tutor docente accede al listado de tareas de los alumnos asignados.
2. El sistema muestra las tareas con estado COMPLETADA.
3. El tutor docente selecciona una tarea.
4. El sistema muestra los detalles de la tarea realizada por el alumno.
5. El tutor docente valida la tarea.
6. El sistema actualiza el estado de la tarea a VALIDADA.
7. El sistema guarda los cambios en la base de datos.
8. El sistema confirma la validación.

Flujos alternativos:

- Si el tutor docente considera que la tarea no es correcta, puede rechazarla (si está contemplado en el sistema), devolviéndola a estado EN_PROGRESO o ASIGNADA.
- Si la tarea no está en estado COMPLETADA, el sistema no permite su validación.
- Si ocurre un error en la actualización, el sistema muestra un mensaje de fallo.

Resultado:

La tarea queda validada correctamente y pasa al estado VALIDADA, quedando registrada la supervisión del tutor docente.

CU07 – Registro de asistencia**Actor principal:** Alumno**Descripción:**

El alumno registra diariamente su jornada de prácticas mediante el fichaje de entrada y salida. El sistema calcula automáticamente las horas trabajadas en cada día y las acumula en el total de horas del alumno.

Flujo principal:

1. El alumno accede al módulo de asistencia.
2. El sistema muestra la opción de registrar entrada o salida según el estado del día.
3. El alumno registra la hora de entrada.
4. El sistema guarda la hora de entrada asociada a la fecha actual.
5. El alumno introduce obligatoriamente observaciones al registrar la salida
6. El alumno registra la hora de salida al finalizar la jornada.
7. El sistema calcula la duración de la jornada en horas.

8. El sistema actualiza las horas trabajadas del día y las horas totales acumuladas del alumno.
9. El sistema guarda el registro de asistencia en la base de datos.

Flujos alternativos:

- Si el alumno introduce una hora manual, el sistema la utiliza en lugar de la hora actual.
- Si ya existe un registro de entrada para el día, el sistema impide duplicarlo.
- Si ya se ha registrado la salida, el sistema no permite modificarla directamente.
- Si no introduce observaciones, el sistema no permite completar el fichaje

Resultado:

El sistema registra correctamente la jornada del alumno y actualiza sus horas totales de prácticas.

CU08 – Seguimiento de asistencia y tareas

Actor principal: Tutor docente

Descripción:

El tutor docente consulta la información de seguimiento de los alumnos asignados, incluyendo su asistencia diaria y el estado de las tareas. Esta funcionalidad permite supervisar la evolución del alumno durante el periodo de prácticas.

Flujo principal:

1. El tutor docente accede al módulo de seguimiento.
2. El sistema muestra la lista de alumnos asignados.
3. El tutor docente selecciona un alumno.
4. El sistema muestra la información del alumno seleccionando:
 - Registro de asistencia (fechas, horas de entrada/salida y horas acumuladas).
 - Lista de tareas asignadas y su estado actual.
5. El tutor docente analiza la evolución del alumno.

Flujos alternativos:

- Si el alumno no tiene registros de asistencia o tareas, el sistema muestra la información disponible o un mensaje informativo.
- Si no existen alumnos asignados, el sistema muestra una lista vacía.

Resultado:

El tutor docente obtiene una visión completa del progreso del alumno en prácticas para su seguimiento académico.

CU09 – Sincronización externa**Actor principal:** Sistema externo (ERP)**Descripción:**

El sistema externo solicita datos al sistema principal para mantener un repositorio espejo actualizado. Esta sincronización permite que el sistema externo disponga de información consistente sobre alumnos, tareas, asistencias y demás entidades relevantes del sistema.

Flujo principal:

1. El sistema externo inicia una solicitud de sincronización.
2. El sistema principal recibe la petición de datos.
3. El sistema principal valida la solicitud.
4. El sistema principal recopila la información requerida (usuarios, tareas, asistencias, etc.).
5. El sistema principal envía los datos al sistema externo.
6. El sistema externo recibe y almacena la información en su repositorio.
7. Se confirma la sincronización entre ambos sistemas.

Flujos alternativos:

- Si la solicitud no es válida, el sistema principal la rechaza.
- Si ocurre un error en la comunicación, la sincronización no se completa y se notifica el fallo.
- Si algunos datos no están disponibles, se envía la información parcial disponible.

Resultado:

El sistema externo mantiene su repositorio actualizado con los datos del sistema principal.

CU10 – Generación de informes PDF**Actor principal:** Sistema externo (ERP / Odoo)**Descripción:**

El sistema externo genera informes en formato PDF a partir de los datos almacenados en su propia base de datos. Estos informes permiten la consulta y exportación de información sobre alumnos, tareas y asistencias sin necesidad de comunicación con el sistema principal.

Flujo principal:

1. El usuario o el sistema solicita la generación de un informe.
2. El sistema externo accede a su base de datos interna.
3. El sistema recupera la información necesaria (alumnos, tareas, asistencias).
4. El sistema procesa los datos obtenidos.
5. El sistema genera el informe en formato PDF.
6. El sistema almacena o permite la descarga del informe generado.

Flujos alternativos:

- Si no hay datos disponibles, el sistema genera un informe vacío o con información parcial.
- Si ocurre un error en la generación del PDF, el sistema cancela el proceso y registra el fallo.

Resultado:

Se genera un informe en PDF a partir de los datos internos del sistema externo, sin dependencia del sistema principal.

CU11 – Predicción de horas

Actor principal: Sistema externo (microservicio de IA)

Descripción:

Al crear una tarea, el sistema invoca un microservicio de inteligencia artificial encargado de estimar la duración aproximada de la tarea. Esta estimación se utiliza como apoyo para la planificación del trabajo del alumno.

Flujo principal:

1. El sistema detecta la creación de una nueva tarea.
2. El sistema envía los datos relevantes de la tarea al microservicio de IA (descripción).
3. El microservicio procesa la información recibida.
4. El microservicio genera una estimación de horas necesarias.
5. El sistema recibe la predicción de horas.
6. El sistema almacena la estimación junto a la tarea creada.

Flujos alternativos:

- Si el microservicio no está disponible, la tarea se crea sin estimación de horas.
- Si la respuesta del microservicio es inválida, el sistema ignora la predicción y continúa el proceso.

Resultado:

La tarea queda registrada con una estimación de horas generada por el microservicio de IA, o sin ella si el servicio no está disponible.

2.3 Actores del sistema

Actores del Sistema					
Profesor Gestor	Tutor Empresa	Alumno	Profesor Tutor	Sistema Externo ERP	Microservicio Predicción

Profesor gestor

Es el usuario encargado de la configuración inicial del sistema. Su función principal es la creación y gestión de las entidades base del sistema, como centros educativos, usuarios (alumnos, tutores docentes y tutores de empresa) y empresas. Además, establece las relaciones entre dichas entidades. No participa en el seguimiento diario de las prácticas.

Tutor de empresa

Es el responsable del seguimiento del alumno en la empresa. Se encarga de crear, asignar y gestionar las tareas del alumno, así como de su modificación o cancelación. También puede consultar el estado de las tareas asignadas.

Alumno

Es el usuario que realiza las prácticas. Su función es registrar su asistencia diaria y gestionar el estado de las tareas asignadas, actualizando su progreso según su evolución (en progreso, completada, etc.).

Profesor Tutor

Es el responsable del seguimiento académico del alumno. Puede consultar la asistencia y el estado de las tareas, y validar las tareas una vez han sido completadas por el alumno.

Sistema externo (ERP / Odoo)

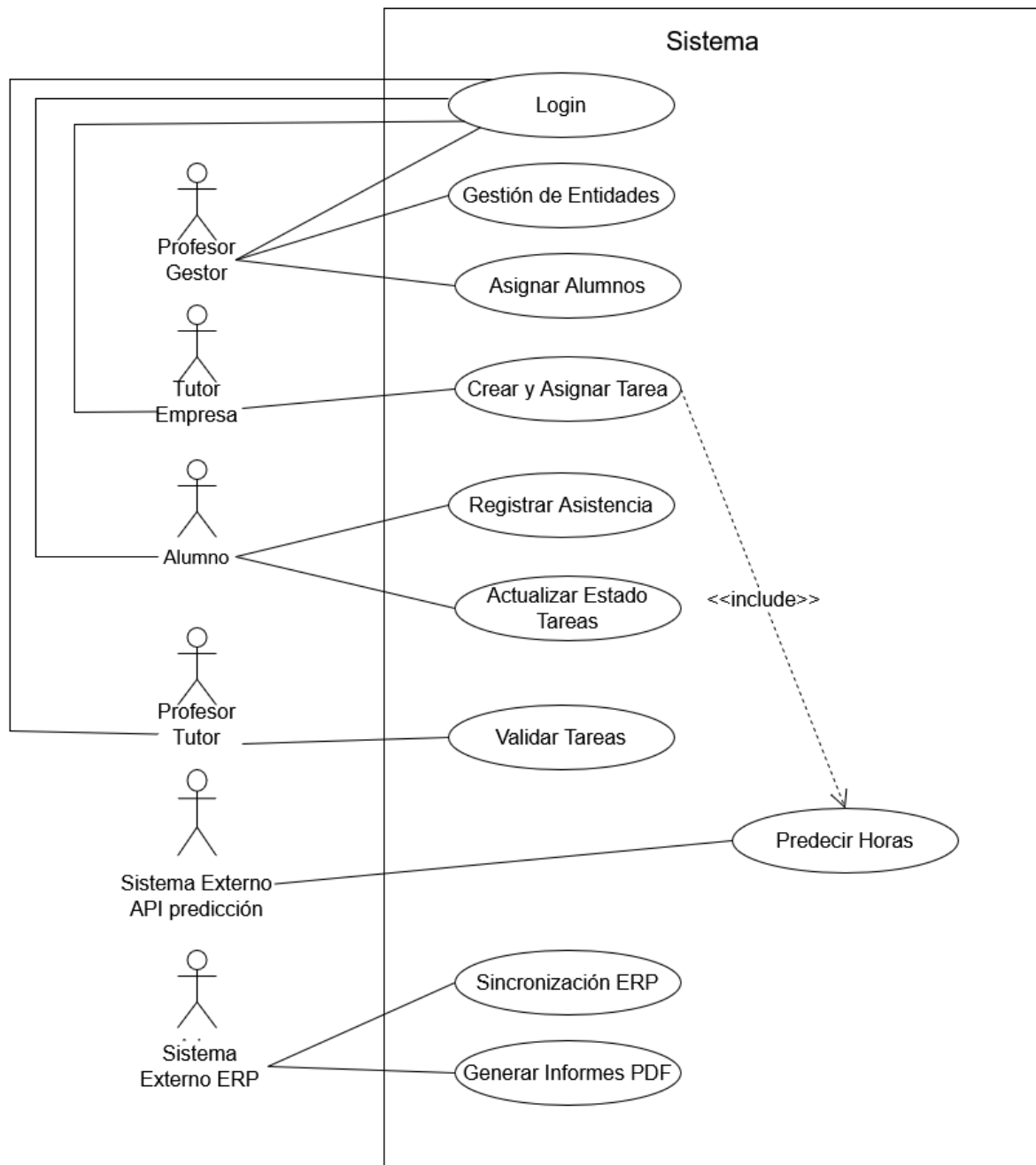
Sistema externo que actúa como repositorio espejo de los datos del sistema principal. Recibe información de alumnos, tutores, tareas y asistencia. Además, puede generar informes en formato PDF y solicitar datos al sistema principal bajo demanda.

Microservicio de predicción (IA)

Servicio externo utilizado para estimar el tiempo necesario para la realización de una tarea. Es invocado automáticamente al crear una tarea, devolviendo una predicción de horas estimadas.

2.4 Diagrama de Casos de Uso

Figura 1. Diagrama de Casos de uso.



2.4 Historias de Usuario

ID	Historia de usuario	Prioridad
HU01	Como alumno, quiero registrar mi asistencia diaria para llevar un control de mis horas de prácticas.	Alta
HU02	Como alumno, quiero actualizar el estado de mis tareas para reflejar mi progreso.	Alta
HU03	Como tutor de empresa, quiero crear tareas para asignarlas a los alumnos.	Alta
HU04	Como tutor de empresa, quiero asignar tareas a alumnos para organizar su trabajo.	Alta
HU05	Como tutor docente, quiero validar tareas completadas para hacer seguimiento académico.	Alta
HU06	Como tutor docente, quiero consultar la asistencia y tareas de mis alumnos para supervisar su evolución.	Media
HU07	Como profesor gestor, quiero gestionar usuarios y entidades para configurar el sistema.	Alta
HU08	Como sistema, quiero estimar automáticamente las horas de una tarea para facilitar su planificación.	Media
HU09	Como sistema externo, quiero obtener datos del sistema para mantener un repositorio sincronizado.	Media

2.5 Requisitos de Información

En este apartado se definen las entidades principales que gestiona el sistema, así como los datos asociados a cada una de ellas y las relaciones existentes entre las mismas.

El sistema se compone de un conjunto de entidades que permiten gestionar el seguimiento de las prácticas en empresa del alumnado.

Entidades del sistema principal

Usuario

Entidad base que representa a los distintos tipos de usuarios del sistema.

Atributos:

- Identificador
- Nombre
- Apellidos
- Email
- Contraseña
- Rol (profesor gestor, tutor docente, tutor de empresa, alumno)

Alumno

Representa al usuario que realiza las prácticas.

Atributos específicos:

- Horas totales acumuladas

Relaciones:

- Pertenece a un centro docente
- Está asignado a un tutor docente
- Está asignado a un tutor de empresa
- Puede tener múltiples tareas
- Puede tener múltiples registros de asistencia

Profesor (Tutor docente / gestor)

Representa al personal docente del sistema.

Atributos específicos:

- Número de alumnos asignados

Relaciones:

- Pertenece a un centro docente
- Supervisa a múltiples alumnos
- Puede validar tareas

Tutor de empresa

Representa al responsable del alumno dentro de la empresa.

Atributos específicos:

- Número de alumnos asignados

Relaciones:

- Pertenece a una empresa
- Supervisa a múltiples alumnos
- Crea y asigna tareas

Empresa

Representa la entidad donde el alumno realiza las prácticas.

Atributos:

- Identificador (CIF)
- Razón social
- Dirección
- Localidad
- Teléfono de contacto
- Email de contacto
- Persona de contacto

Relaciones:

- Puede tener varios tutores de empresa
- Puede tener varios alumnos asociados

Centro docente

Representa el centro educativo al que pertenece el alumno.

Atributos:

- Identificador del centro
- Nombre
- Dirección
- Localidad
- Teléfono
- Correo institucional

Relaciones:

- Tiene múltiples profesores
- Tiene múltiples alumnos

Tarea

Representa una actividad asignada al alumno durante sus prácticas.

Atributos:

- Identificador
- Título
- Descripción
- Fecha de asignación
- Fecha límite
- Estado (asignada, en progreso, completada, validada, cancelada)
- Horas estimadas (calculadas automáticamente)
- Horas reales

Relaciones:

- Es creada por un tutor de empresa
- Es asignada a un alumno
- Es validada por un tutor docente

Asistencia

Representa el registro diario de jornada del alumno.

Atributos:

- Identificador
- Fecha
- Hora de entrada
- Hora de salida
- Horas trabajadas
- Observaciones

Relaciones:

- Pertenece a un alumno

Sistema externo (ERP)

El sistema se integra con un sistema externo de gestión empresarial que actúa como repositorio espejo.

Este sistema:

- Recibe información de alumnos, profesores, tareas y asistencias
- Permite la generación de informes en formato PDF
- Puede solicitar datos al sistema principal bajo demanda

No todas las entidades del sistema principal tienen representación en el sistema externo, como es el caso de la empresa o el tutor de empresa.

Servicio externo de predicción

El sistema utiliza un servicio externo para estimar el número de horas necesarias para la realización de una tarea.

Este servicio:

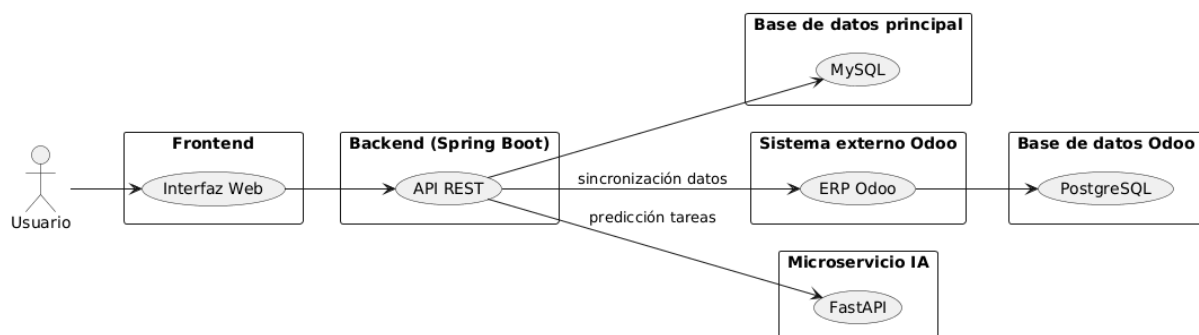
- Es invocado automáticamente durante la creación de tareas
- Devuelve una estimación de horas basada en modelos predictivos

3. Diseño

3.1 Arquitectura del Sistema

Diagrama Arquitectura de Sistemas

Figura 2. Diagrama de Arquitectura de Sistemas.



El sistema desarrollado sigue una arquitectura basada en un modelo cliente-servidor con integración de servicios externos, donde el núcleo funcional se encuentra en una API REST desarrollada con Spring Boot.

La arquitectura está compuesta por varios subsistemas independientes que interactúan entre sí:

Sistema principal (Backend - Spring Boot)

El backend constituye el núcleo del sistema y es responsable de la lógica de negocio, la gestión de usuarios, tareas y asistencias, así como del control de acceso mediante autenticación.

Este componente expone una API REST que es consumida por el frontend y por sistemas externos.

Además, gestiona la persistencia de datos en una base de datos relacional MySQL, que actúa como fuente de verdad del sistema.

Frontend

El frontend proporciona la interfaz de usuario del sistema, permitiendo la interacción de los distintos actores (alumnos, tutores y profesores).

Se comunica exclusivamente con el backend mediante peticiones HTTP a la API REST.

Sistema externo ERP (Odoo)

Se integra un sistema ERP externo basado en Odoo que actúa como repositorio espejo de la información del sistema principal.

Este sistema:

- Consume datos desde la API del backend
- Almacena información en su propia base de datos PostgreSQL
- Permite la visualización de datos y la generación de informes en formato PDF

Odoo no modifica ni envía información al sistema principal, por lo que no se considera una fuente de verdad, sino un sistema de soporte orientado a explotación de datos.

Microservicio de predicción (FastAPI)

El sistema incorpora un microservicio desarrollado en FastAPI encargado de estimar la duración de las tareas mediante modelos de predicción.

Este servicio:

- Es invocado por el backend durante la creación de tareas
- Recibe datos de la tarea
- Devuelve una estimación de horas
- No almacena información persistente

Bases de datos

El sistema utiliza dos bases de datos diferenciadas:

- **MySQL**: base de datos principal del sistema
- **PostgreSQL**: base de datos del sistema Odoo

Despliegue y orquestación

Todos los componentes del sistema se despliegan de forma conjunta mediante Docker Compose, lo que permite:

- La ejecución aislada de cada servicio
- La comunicación entre servicios a través de una red interna
- La portabilidad del sistema entre diferentes entornos

3.3 Diagramas de Secuencia

Figura 4. Diagrama de Secuencia Asistencia

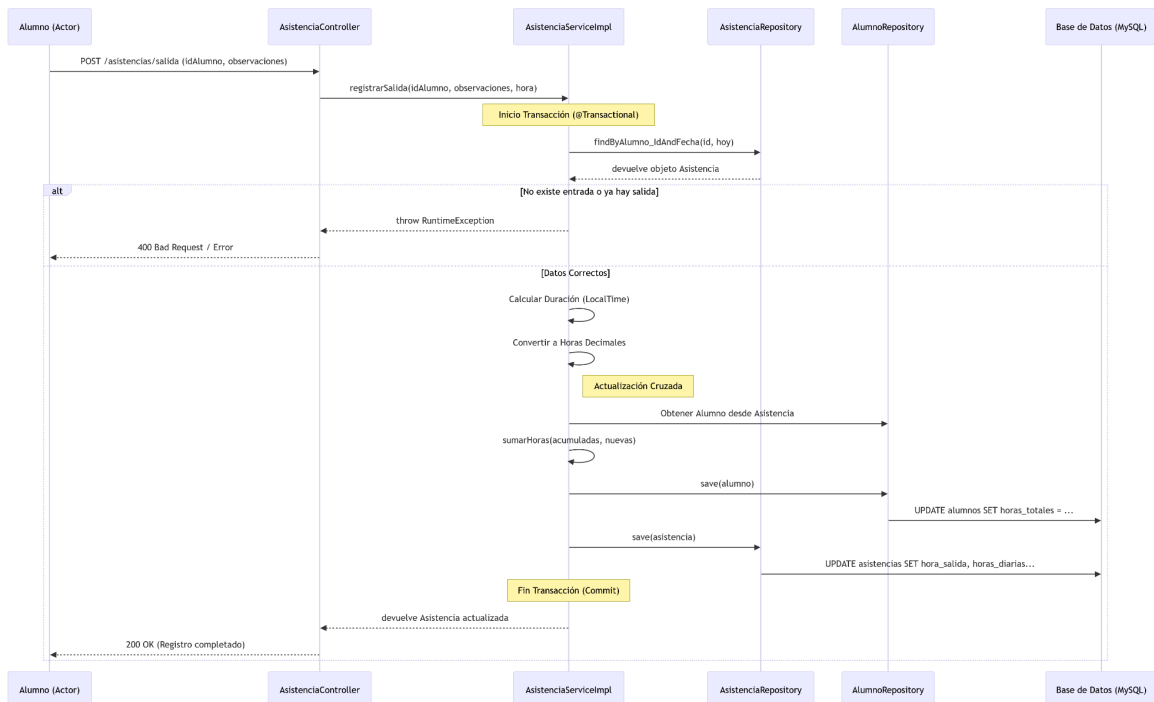


Figura 5. Diagrama de Secuencia Predicción Horas

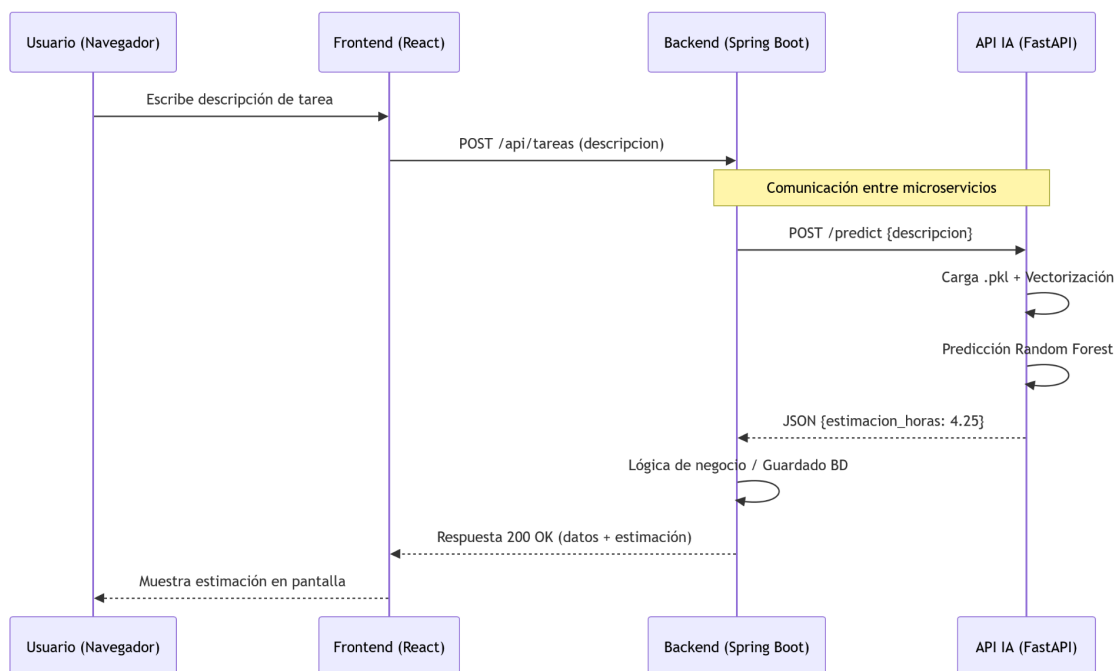
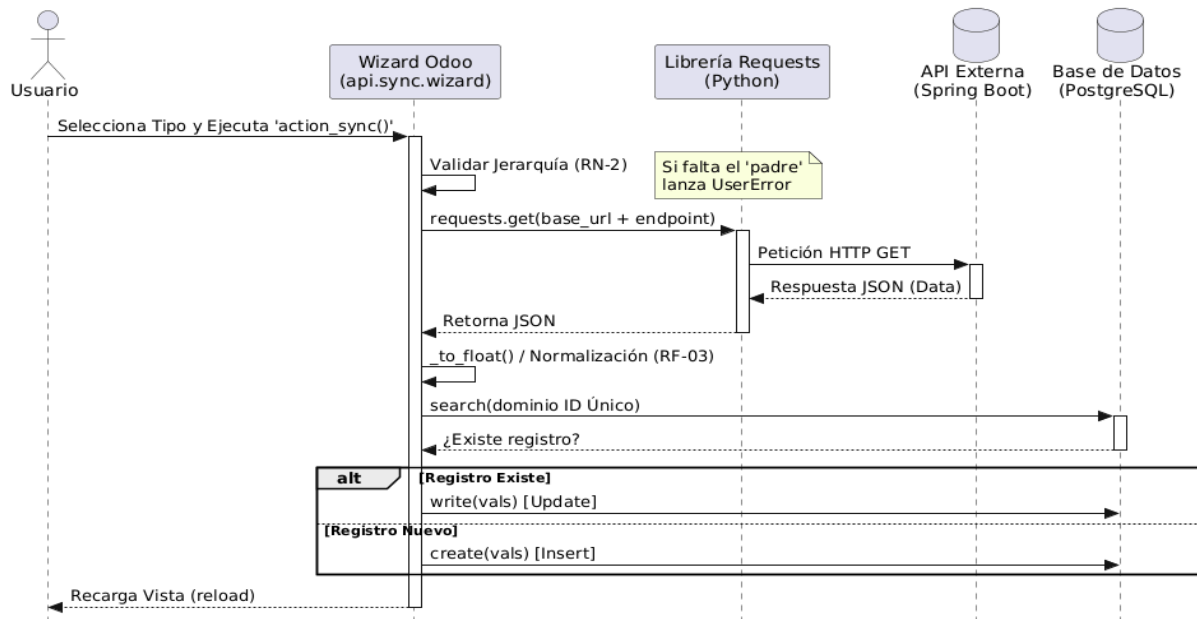
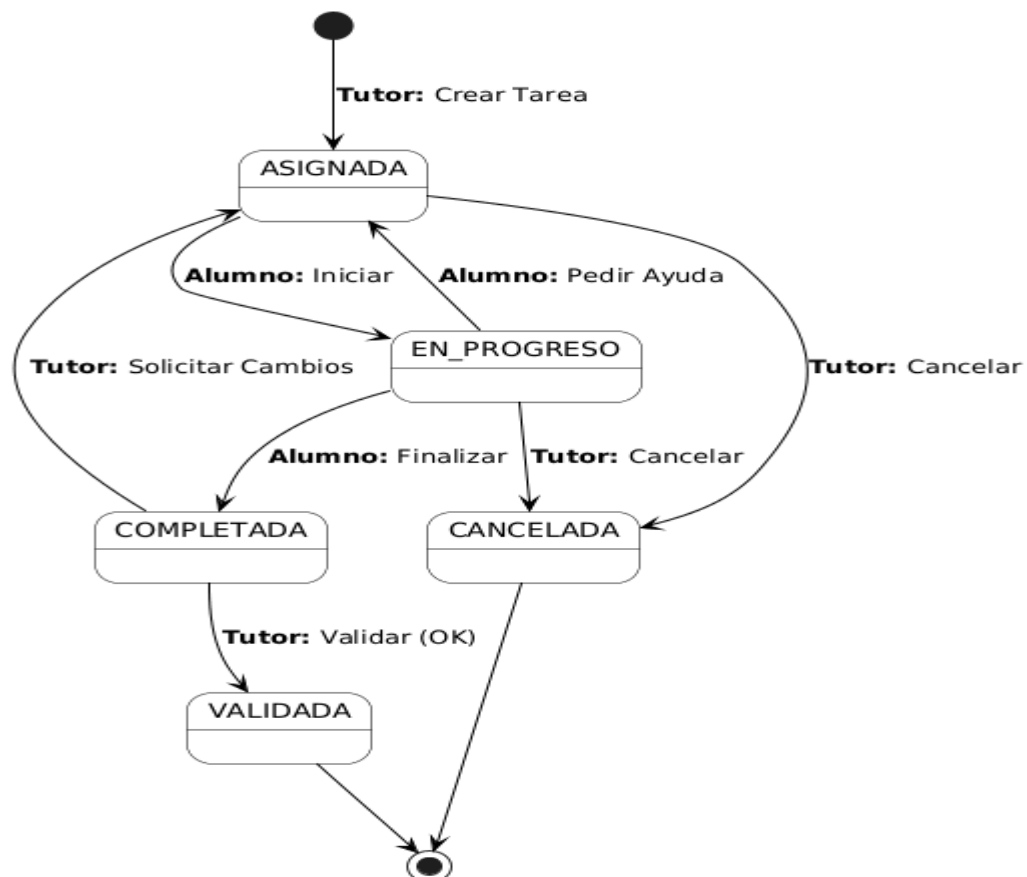


Figura 6. Diagrama de Secuencia Sincronización con Spring



3.4 Diagramas de Estado

Figura 7. Diagrama de Estado Tarea



4. Tecnologías y Entornos de desarrollo

Para el desarrollo de este proyecto, he optado por una arquitectura modular que separa claramente el frontend, el backend y los servicios externos. El objetivo principal ha sido crear un sistema robusto pero fácil de mantener, utilizando contenedores para que todo el ecosistema (Base de datos, API, IA y ERP) se pueda levantar de forma conjunta sin complicaciones de configuración.

4.1 Stack tecnológico

A continuación, detallo las tecnologías que he elegido para cada parte del sistema:

- **Frontend (Interfaz de usuario):** He utilizado **React 19** junto con **Vite 8** para conseguir un entorno de desarrollo rápido y ligero. Para el diseño visual y que la aplicación sea cómoda de usar en distintos dispositivos, he integrado **Bootstrap 5.3** y **React-Bootstrap**. Las peticiones a la API del backend se gestionan mediante la librería **Axios**.
- **Backend Principal:** Está desarrollado con **Java 17** y el framework **Spring Boot 3.5.11**. He usado **Maven** para gestionar todas las dependencias y **Spring Data JPA** para que la comunicación con la base de datos sea más sencilla mediante repositorios. Para optimizar el desarrollo en Java, se ha integrado la librería **Lombok**, que permite automatizar la generación de constructores, getters, setters y otros métodos repetitivos mediante anotaciones, manteniendo el código más limpio y legible.
- **Microservicio de IA:** Para la predicción de horas de las tareas, he creado un servicio independiente con **FastAPI** y **Python 3.11-slim**. He elegido esta tecnología porque permite cargar y ejecutar modelos de Machine Learning (usando librerías como scikit-learn y pandas) de una manera mucho más eficiente que en Java.
- **ERP Externo (Odoo):** Se utiliza **Odoo 17** funcionando sobre **PostgreSQL 15**. He desarrollado un módulo propio en Python para que Odoo pueda conectarse a la API de Spring y traerse los datos de los alumnos y las asistencias.
- **Bases de Datos:** El corazón de la aplicación guarda los datos en **MySQL 8.0**, mientras que Odoo gestiona su propia información en **PostgreSQL**.
- **Infraestructura y Despliegue:** Todo el sistema se orquesta con **Docker (versión 28.3)**. Aunque el desarrollo se hizo entre Windows y máquinas virtuales Ubuntu (VirtualBox), el proyecto final está preparado para ejecutarse de forma unificada.

4.2 Modelo físico de datos (BBDD)

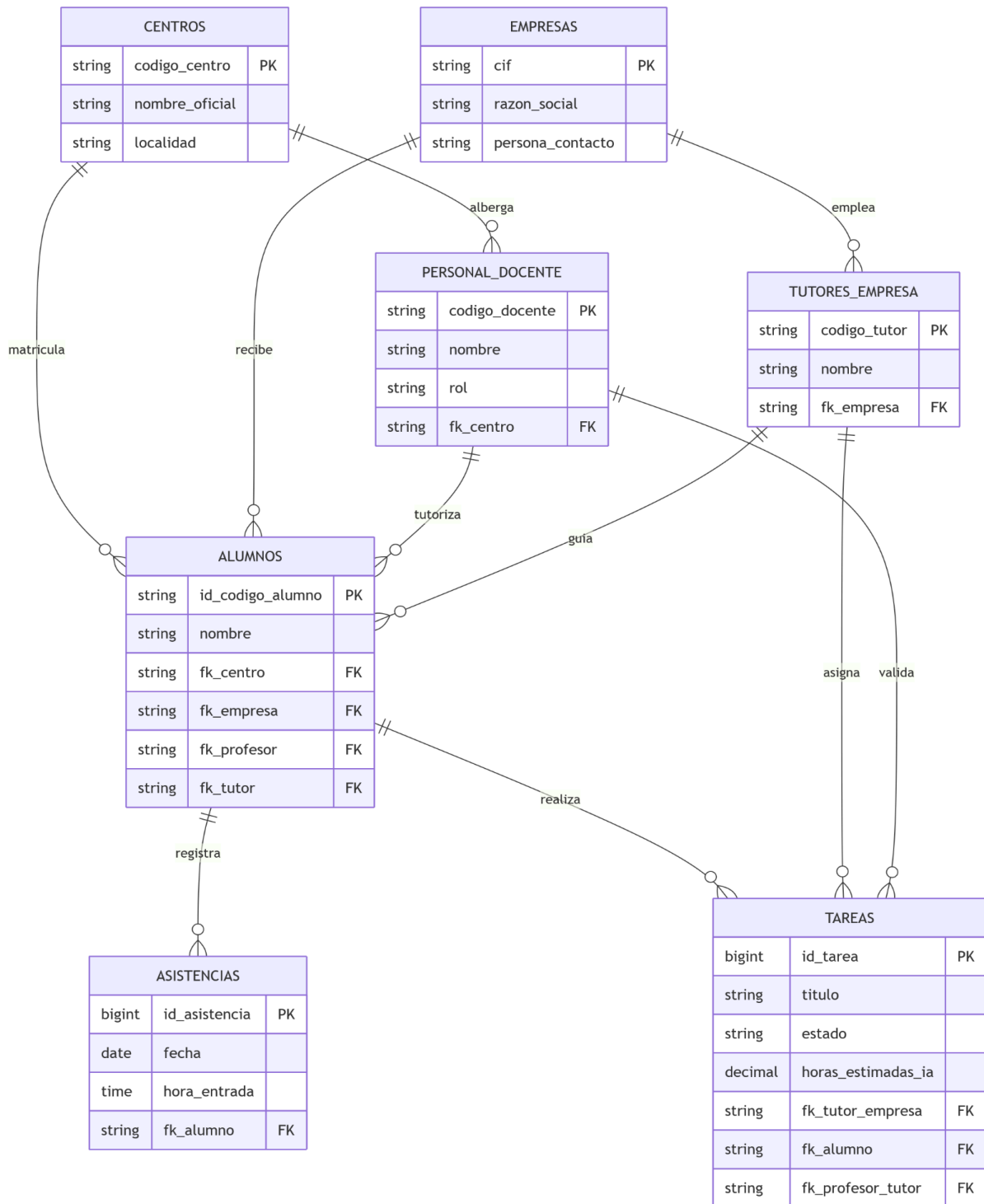
La persistencia de la aplicación se apoya en dos bases de datos diferenciadas por su propósito. La base de datos principal, gestionada con **MySQL 8.0**, almacena toda la lógica de negocio y la información generada por los usuarios de la app. Por otro lado, **PostgreSQL 15** actúa como el motor de almacenamiento de Odoo 17, manteniendo una copia sincronizada de los datos necesarios para la generación de informes.

Para el diseño del esquema en MySQL, he seguido una estructura relacional donde las entidades se conectan mediante claves foráneas para asegurar que no haya datos huérfanos. Algunos detalles clave del diseño son:

- **Entidad Alumno:** Almacena las horas_totales como un tipo INT, ya que la lógica del backend redondea el total acumulado al finalizar cada jornada.
- **Entidad Asistencia:** Utiliza tipos TIME para las entradas y salidas, y un DOUBLE para horas_diarias, permitiendo así registrar con precisión fracciones de hora (ej. 4.5 horas).
- **Entidad Tarea:** Incluye el campo horas_estimadas_ia que se rellena automáticamente tras la respuesta del microservicio FastAPI.
- **Gestión de Roles:** El rol de cada usuario se almacena como una cadena de texto que el backend valida para permitir o denegar el acceso a diferentes partes de la interfaz.

Diagrama Entidad-Relación

Figura 8. Diagrama entidad relación bbdd (MySQL) FTC Gestión



Diccionario de Datos (Selección de tablas principales)

Para complementar el diagrama anterior, se detallan a continuación las especificaciones técnicas de las tablas que gestionan la lógica principal del sistema:

Tabla: Alumnos

Representa a los estudiantes matriculados y sus relaciones con el centro y la empresa.

Campo	Tipo	Nulo	Descripción
id_codigo_alumno	VARCHAR(20)	NO	Clave Primaria. Identificador único del alumno.
email	VARCHAR(100)	NO	Correo electrónico (Unique).
horas_totales	INT	SÍ	Acumulado de horas redondeadas.
fk_centro	VARCHAR(20)	SÍ	Clave foránea que apunta a la tabla Centros.
fk_profesor	VARCHAR(20)	SÍ	Clave foránea que vincula con su tutor docente.

Tabla: Asistencias

Registra la jornada diaria y el cómputo de horas de cada alumno.

Campo	Tipo	Nulo	Descripción
id_asistencia	BIGINT	NO	Clave Primaria (Auto_increment).
fecha	DATE	NO	Fecha del registro de asistencia.
hora_entrada	TIME	NO	Hora de inicio de la jornada.
horas_diarias	DOUBLE	SÍ	Cálculo decimal de las horas trabajadas en el día.
fk_alumno	VARCHAR(20)	NO	Clave foránea (Relación 1:N con Alumnos).

Tabla: Tareas

Gestiona las actividades asignadas y la interacción con el microservicio de IA.

Campo	Tipo	Nulo	Descripción
id_tarea	BIGINT	NO	Clave Primaria (Auto_increment).
estado	VARCHAR(20)	SÍ	Estado actual (ASIGNADA, COMPLETADA,..)
horas_estimadas_ia	DECIMAL(5,2)	SÍ	Predicción generada por el microservicio FastAPI.
fk_tutor_empresa	VARCHAR(20)	NO	Clave foránea del tutor que asigna la tarea.

Otras entidades

El resto de entidades del sistema, como **Personal Docente** y **Tutores de Empresa**, completan la estructura de usuarios y permisos de la aplicación. Estas tablas son fundamentales para la trazabilidad de las tareas, ya que actúan como claves foráneas en la entidad Tareas para identificar quién asigna y quién valida cada actividad. Sus atributos y relaciones detalladas (como la vinculación obligatoria de cada docente a un Centro o cada tutor a una Empresa) pueden consultarse de forma gráfica en el diagrama de entidad-relación superior.

Del mismo modo, las entidades **Centros** y **Empresas** funcionan como la base organizativa del modelo físico, ya que centralizan los datos de ubicación y contacto de las instituciones donde se desarrollan las prácticas, permitiendo que el resto de tablas mantengan la integridad referencial necesaria para el correcto funcionamiento del sistema.

5. Implementación

Una vez definidas las tecnologías y el modelo de datos, en este apartado se describe el proceso de construcción del ecosistema **FTC Gestión**. La implementación se ha abordado desde una perspectiva de microservicios y modularidad, asegurando que cada componente (Backend, Frontend, IA y ERP) cumpla una función específica y se comunique con el resto de forma eficiente. A continuación, se detallan los retos técnicos superados y la lógica aplicada en el despliegue y desarrollo de cada módulo.

5.1 Configuración del Entorno y Orquestación (Docker)

Para garantizar la estabilidad del sistema FTC Gestión, se ha diseñado una infraestructura basada en contenedores utilizando **Docker Compose**. Esta aproximación permite que todos los módulos (Frontend, Backend, IA y ERP) coexistan en un entorno aislado, replicable y fácil de desplegar.

Configuración del Dockerfile en el Frontend

Una de las particularidades de este proyecto es la gestión del **Frontend**. A diferencia de otros despliegues que utilizan imágenes estáticas, el **Dockerfile** de React está configurado para realizar el **build** y la ejecución directamente desde el contexto del código fuente. Esto asegura que cualquier cambio en la interfaz se integre en la imagen durante la fase de construcción, permitiendo un control total sobre el entorno de ejecución de Vite.

Red Global y Resolución de Nombres

Se ha implementado una red interna denominada **red-global-dam** con el driver **bridge**. Esta red actúa como un servidor DNS propio para el proyecto:

- **Sustitución de Localhost:** Al estar todos los servicios en la misma red, se han sustituido las direcciones locales por los nombres de servicio definidos en el archivo YAML (como db-mysql o api-ml).

- **Orden de arranque:** Mediante la instrucción **depends_on**, se ha establecido una jerarquía de encendido. Por ejemplo, el Backend (api-spring) espera a que la base de datos (db-mysql) esté operativa antes de intentar la conexión, evitando fallos críticos durante el despliegue.

Persistencia y Gestión de Volúmenes

La seguridad de los datos se gestiona mediante volúmenes que conectan el almacenamiento del contenedor con el disco físico del host:

- **Base de Datos Principal:** El volumen **mysql-data** asegura que la información de alumnos y tareas no se pierda al reiniciar los servicios. Además, se utiliza la ruta **docker-entrpoint-initdb.d** para cargar automáticamente el esquema inicial de la base de datos.
- **Integración con Odoo:** Se han mapeado carpetas locales para los datos de PostgreSQL y los **addons** personalizados del ERP, permitiendo que el módulo de sincronización sea persistente y editable.

Resolución de Conflictos y Desafíos Técnicos

Durante el desarrollo se detectaron varios desafíos que marcaron la evolución de la infraestructura:

1. **Conflicto de Caché en el Backend:** Se identificó que Docker solía reutilizar capas antiguas de las imágenes, lo que impedía que las nuevas correcciones en el código Java se reflejaran tras el despliegue. Para solucionar esto, se estableció un procedimiento de limpieza que incluye el borrado de volúmenes y la reconstrucción sin caché (**--no-cache**), asegurando la integridad del código ejecutado.
2. **Migración desde Entornos Virtuales:** Inicialmente, Odoo operaba desde una máquina virtual externa. La conectividad se gestionaba mediante la IP del host, lo que generaba inestabilidad. Al integrar Odoo en el **docker-compose.yml**, se simplificó la arquitectura, permitiendo que el sistema se comunique de forma nativa a través de los nombres de los contenedores.

Resumen de Servicios Orquestados

El sistema queda compuesto por seis nodos principales:

- **react-frontend-dam:** Interfaz de usuario en el puerto 5173.
- **api-spring-core:** Backend principal en Java 17.
- **mysql_dam_project:** Motor de base de datos MySQL 8.0.
- **api-ml-python:** Microservicio de IA basado en FastAPI.
- **odoo_sge:** ERP Odoo 17.0.
- **db-postgres-odoo:** Base de datos PostgreSQL para el soporte del ERP.

5.2 Desarrollo del Backend (Spring Boot)

El núcleo del sistema se ha desarrollado utilizando **Java 17** y **Spring Boot 3.5.11**. La arquitectura se basa en una estructura de capas que facilita el mantenimiento y la escalabilidad, asegurando que la lógica de negocio esté claramente separada de la persistencia y de los puntos de entrada de la API.

Arquitectura y Estructura de Paquetes

Se ha seguido el patrón de diseño recomendado por la comunidad de Spring, organizando el código en los siguientes paquetes principales:

- **Controller:** Define los endpoints de la API y gestiona las peticiones HTTP entrantes.
- **Service:** Contiene la lógica de negocio. Se ha optado por un diseño basado en interfaces (**Service**) y sus correspondientes implementaciones (**ServiceImpl**), lo que permite una mayor flexibilidad y facilita las pruebas unitarias.
- **Repository:** Interfaces que extienden de **JpaRepository**. Al utilizar Spring Data JPA, la implementación de las consultas a la base de datos MySQL es gestionada automáticamente por el framework, optimizando el tiempo de desarrollo.
- **Entity:** Modelos de datos que representan las tablas físicas de la base de datos MySQL mediante anotaciones de JPA.

Interfaz de Programación de Aplicaciones (API REST)

Para permitir la comunicación entre el Frontend, el microservicio de IA y el ERP Odoo, se ha diseñado una API siguiendo los principios REST. Los recursos se exponen mediante endpoints protegidos por lógica de negocio que aseguran que cada rol acceda únicamente a su información permitida.

A continuación, se presenta una tabla resumen con los puntos de entrada más significativos del sistema, que sirven como base para la integración de todos los módulos:

Tabla Endpoints Spring		
Método	Endpoint	Propósito en la Aplicación
GET	/api/centros/principal	Carga los datos y el logo del centro en la cabecera.
POST	/api/asistencias/entrada/{id}	Registra el inicio de jornada del alumno.
PATCH	/api/asistencias/salida/{id}	Finaliza la jornada y calcula las horas diarias.
POST	/api/tareas	Creación de nuevas tareas por el Tutor de Empresa.

PATCH	/api/tareas/{id}/estado	El alumno marca la tarea como completada.
GET	/api/alumnos/huerfanos	Panel del Gestor para detectar alumnos sin tutor.
GET	/api/tutores-empresa/{id}/alumnos	Listado de alumnos bajo supervisión de un tutor.

Estandarización de respuestas

Todos los endpoints devuelven objetos en formato JSON. Se han utilizado los códigos de estado HTTP estándar (200 OK, 201 Created, 404 Not Found, etc.) para que el Frontend y Odoo puedan gestionar los errores de forma predecible.

Documentación y Pruebas

Durante el desarrollo, la validez de estos puntos de entrada se ha contrastado mediante herramientas de testeo de APIs (como Postman), asegurando que los parámetros enviados desde el Wizard de Odoo o los formularios de React reciban la respuesta esperada.

Transferencia de Datos y Mapeo (DTO y Mappers)

Para garantizar una comunicación segura y eficiente entre el Backend y el Frontend, se ha implementado el patrón **DTO (Data Transfer Object)**. En lugar de enviar las entidades JPA completas a través de la API, se han diseñado objetos específicos que contienen únicamente la información necesaria para cada vista de la interfaz.

- **Seguridad y Desacoplamiento:** El uso de DTO evita exponer campos sensibles de la base de datos y permite que el modelo físico pueda evolucionar sin afectar directamente al contrato de la API.
- **Capa de Mapeo (Mappers):** Para la conversión entre Entidades y DTO, se han desarrollado clases de mapeo. Estas clases se encargan de transformar los objetos de persistencia en objetos de transferencia de forma limpia, centralizando la lógica de conversión y evitando la dispersión de este código en los servicios.

Comunicación con el Microservicio de IA

La integración con el motor de predicción en Python se realiza mediante la clase **IARestClient**. Este componente utiliza **RestTemplate** para consumir la API de FastAPI de forma síncrona:

- **Flujo de Trabajo:** Cuando un Tutor de Empresa crea una nueva tarea, el servicio de tareas invoca automáticamente al cliente de IA enviando la descripción.
- **Resiliencia:** Se ha implementado un bloque **try-catch** que captura cualquier excepción en la red (por ejemplo, si el contenedor de la IA está caído). En caso de error, el sistema asigna un valor por defecto de 0.0 horas, garantizando que el usuario pueda seguir trabajando sin bloqueos.

- **Configuración:** La URL del microservicio no está escrita directamente en el código, sino que se inyecta desde el archivo de propiedades utilizando la anotación **@Value**, lo que facilita el cambio entre entornos de desarrollo y producción.

Lógica de Asistencias y Gestión de Datos

La aplicación actúa como la **fuentes de verdad** única para todo el sistema. Esto significa que los datos de alumnos y asistencias nacen y se validan en la API de Spring antes de ser consultados por otros servicios como Odoo.

- **Cálculo de Tiempos:** El cálculo de la jornada se realiza en el servidor, restando las horas de entrada y salida para obtener las horas diarias en formato **Double**.
- **Integridad de Datos:** Se ha documentado la discrepancia técnica entre el almacenamiento de horas diarias (**Double**) y el acumulado de horas totales del alumno (**Integer**). Esta decisión, aunque detectada como una limitación de precisión durante las pruebas, se ha mantenido para asegurar la compatibilidad con el sistema de redondeo de jornadas completas exigido por el centro.

Desafíos Técnicos y Resolución de Errores (CORS)

Uno de los retos más significativos durante la fase de integración con el Frontend fue la gestión de las políticas de intercambio de recursos de origen cruzado (**CORS**).

- **Problema de Navegador:** Se detectó una inconsistencia crítica donde las peticiones de tipo **PATCH** funcionaban correctamente en Firefox pero fallaban sistemáticamente en Google Chrome, bloqueando la actualización de estados de las tareas.
- **Solución:** Tras una investigación exhaustiva, se identificó que las políticas de seguridad y la gestión de caché de Chrome interferían con las cabeceras de las peticiones. Este aprendizaje permitió ajustar la configuración de seguridad del backend y validar el uso de modos de navegación de incógnito para pruebas de desarrollo, ahorrando tiempos de depuración en versiones futuras.

Autenticación y Gestión de Roles

A falta de una implementación completa de Spring Security, el control de acceso se basa en un sistema de validación por **Email, Password y Rol**.

- **Validación en Login:** El usuario debe seleccionar su perfil (Alumno, TutorEmpresa, etc...) en la interfaz. El Backend recibe estos parámetros y contrasta la información con la tabla correspondiente en la base de datos.
- **Control de Flujo:** El sistema diferencia las acciones permitidas según el rol; por ejemplo, solo un Tutor de Empresa tiene permisos para validar una tarea o asignar una predicción de horas inicial.

5.3 El Microservicio de Predicción (FastAPI + ML)

Para que el sistema sea capaz de predecir cuánto va a tardar una tarea, he creado un pequeño servicio independiente usando **Python**. He decidido hacerlo así porque Python tiene librerías muy potentes para estas cosas que en Java serían mucho más difíciles de programar.

Creación del modelo de predicción

Para que la IA aprenda, he tenido que "entrenarla". Estos son los pasos que seguí:

- **El Dataset:** Creé un archivo con unas 2.000 filas de ejemplos de tareas. Durante la carga, detecté que el uso de comas dentro de las descripciones provocaba errores de lectura. Para solucionarlo, realicé un tratamiento de datos sustituyendo el separador estándar por el **punto y coma (;)**. Finalmente, dividí los datos: el 80% para entrenamiento y el 20% para test (evaluación).
- **La técnica (TF-IDF):** Como la IA no entiende palabras, usé esta técnica para convertir el texto en números. Básicamente, cuenta qué palabras son más importantes en cada frase para saber de qué trata la tarea.
- **El algoritmo:** Probé varios métodos y al final me quedé con uno llamado **Random Forest**. Me di cuenta de que era el más fiable para este proyecto porque fallaba menos que los demás.

Figura 9. Diagrama Comparativa Modelos Entrenados

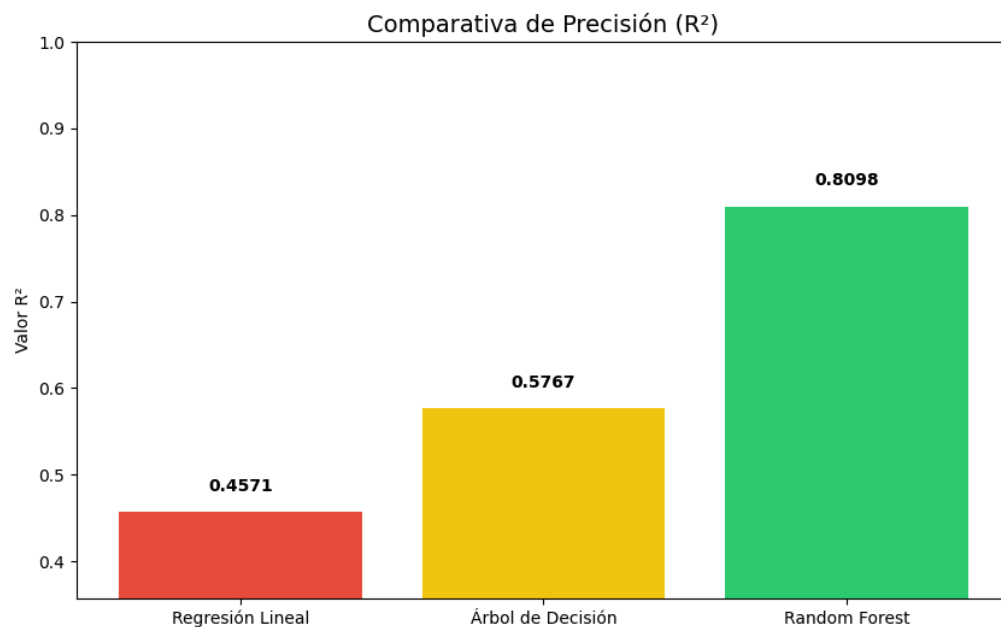
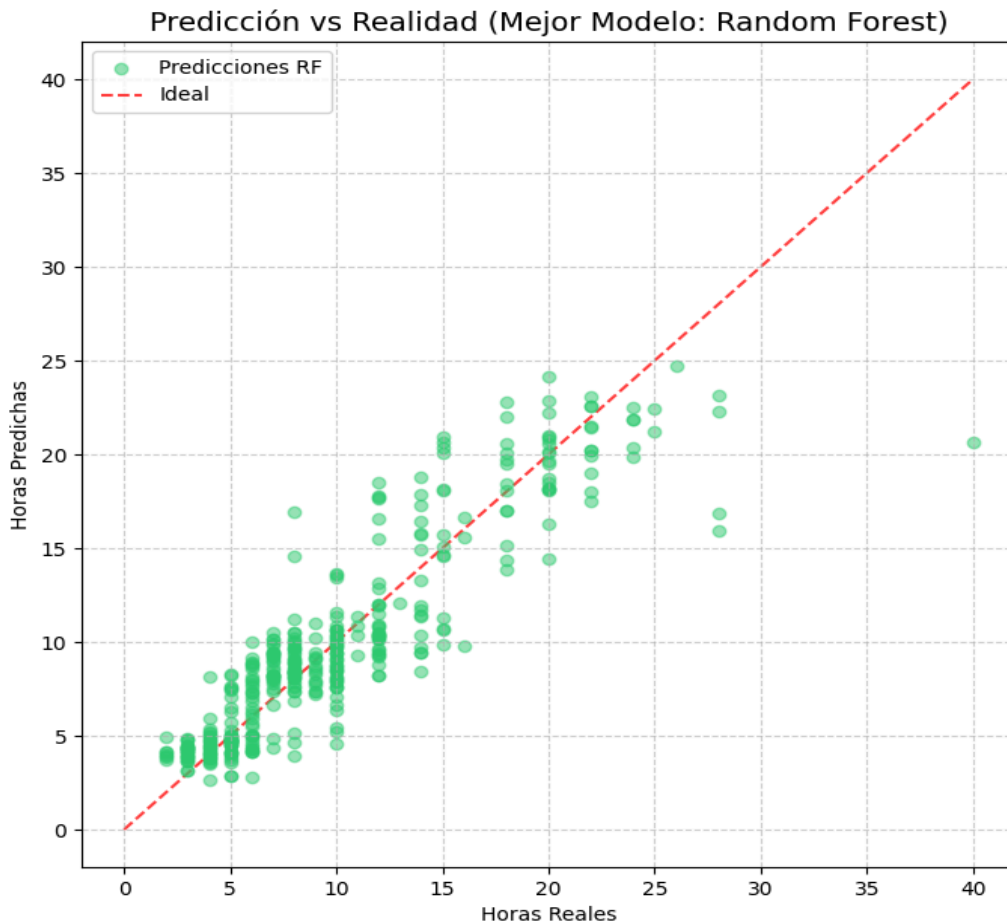


Figura 10. Gráfico de Dispersión de Modelo RandomForest



El truco de los selectores (Ingeniería de datos)

Durante las pruebas vi que, si dejaba que el usuario escribiera cualquier cosa, la IA se confundía mucho y solo acertaba un 40% de las veces.

- **La solución:** Obligué a que todas las descripciones empiecen por tres etiquetas fijas (Tecnología, Tipo de tarea y Dificultad).
- **Resultado:** Para facilitar esto al usuario y no obligarle a escribir códigos raros, puse tres selectores en el Frontend. Gracias a esto, la precisión de la IA subió hasta el **80%**, porque ahora los datos siempre le llegan con la estructura que ella conoce.

Funcionamiento técnico

Para que este servicio esté disponible para el resto de la aplicación, usé un framework llamado **FastAPI**:

- **Archivos .pkl:** Una vez entrenada la IA, utilicé la librería joblib para generar dos archivos: modelo.pkl (el algoritmo entrenado) y vectorizador.pkl (el procesador de texto). De este modo, al arrancar el contenedor de Docker, el servicio carga estos archivos y está listo para funcionar sin necesidad de reentrenar.

- **Conexión:** Este servicio está siempre escuchando. Cuando el Backend de Java le manda una descripción, la IA le devuelve el número de horas estimadas casi al instante.

Figura 11. Swagger de fastAPI



5.4 Integración con Odoo (Módulo Custom)

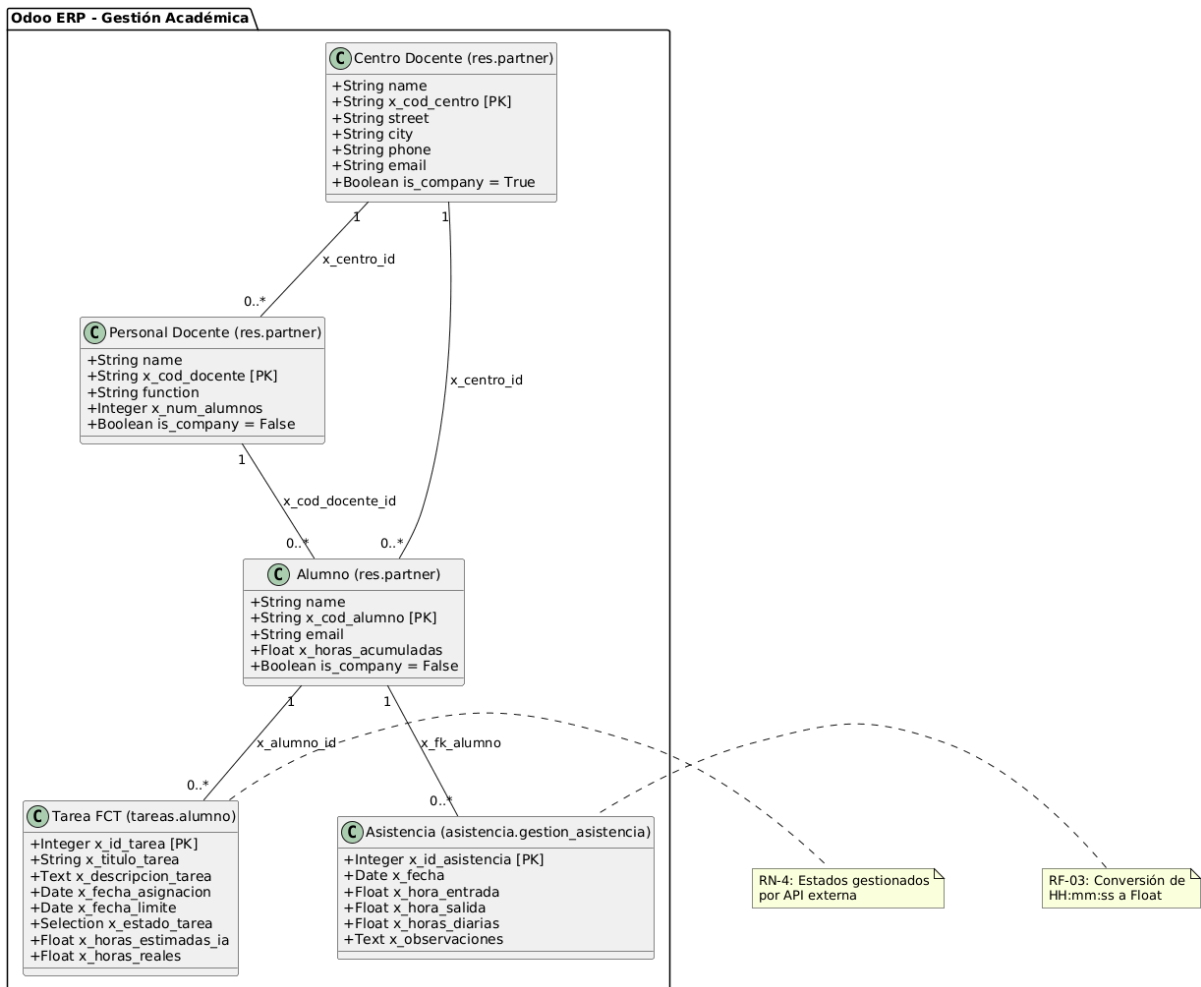
Para la gestión administrativa, se ha desarrollado una solución modular dentro de Odoo. Toda la arquitectura parte de un ítem raíz en el menú principal llamado **Gestión Académica**. Bajo este nodo, se despliegan de forma nivelada el resto de módulos específicos que permiten organizar la información del centro.

Estructura de Módulos y Organización

A diferencia de una instalación estándar, se han creado módulos propios para cada área de gestión, situados en la carpeta **addons**. Estos módulos se presentan al usuario bajo los siguientes nombres:

- **Centro Docente:** Actúa como el núcleo del sistema, donde se define la configuración principal.
- **Personal Docente y Alumnos:** Gestión de los perfiles de usuarios y estudiantes.
- **Tareas FCT y Asistencias:** Módulos dedicados al seguimiento del trabajo diario.
- **Gestión de Informes:** Módulo independiente para la salida de datos.
- **Sincronización con Spring:** Módulo encargado de la comunicación técnica.

Figura 12. Diagrama de Clase en Odoo



El Asistente de Sincronización (Wizard)

La comunicación con el backend de Java no es automática, sino que se gestiona mediante el módulo de **Sincronización con Spring**.

- **Lógica de Importación:** He diseñado un Wizard (ventana modal) que permite al usuario elegir qué entidad desea actualizar. El sistema utiliza la librería requests de Python para consultar la API de Spring Boot.
- **Gestión de Dependencias:** El asistente guía al usuario en el orden lógico. Por ejemplo, el selector obliga a vincular un Centro Docente antes de importar un Alumno, evitando errores de integridad. Para asegurar la asociación entre sistemas, cada registro en Odoo almacena el ID original de la base de datos de Spring.
- **Herencia y Diferenciación:** Se ha heredado de res.partner para aprovechar la base de Odoo, usando el campo is_company para distinguir entre el Centro Docente (True) y el resto de personal (False). En lugar de usar IDs complejos para las relaciones iniciales, se ha utilizado el campo name para facilitar la carga de datos.

Figura 13. Asistente para la sincronización de datos con Spring

Asistente de Sincronización [X]

Tipo de Sincronización

- ☐ Centro Docente
- ☐ Personal Docente
- ☒ Alumnos
- ☐ Asistencias
- ☐ Tareas FCT

Centro Docente

IES Innovación Sevilla

Buscar más...

Empiece a escribir...

Iniciar Sincronización

Generación de Informes PDF

Para cubrir la necesidad de documentación física y oficial, se ha desarrollado el módulo **Gestión de Informes**.

- **Tecnología QWeb:** Se han diseñado plantillas específicas utilizando el motor de informes de Odoo (QWeb), lo que permite generar documentos PDF profesionales a partir de los datos de **Asistencias** y **Tareas FCT**.
- **Automatización:** El módulo permite extraer resúmenes mensuales de las horas realizadas por cada alumno, listos para ser firmados por los tutores.

Figura 14. Asistente para la generación de informes PDF

Generar Informes [X]

Tipo de Informe

- ☒ Informe de Alumno
- ☐ Informe de Centro Docente

Alumno

Alumno Catorce

Alumno Cinco

Alumno Cuatro

Alumno Diecinueve

Alumno Dieciocho

Alumno Diecisiete

Generar

Figura 15. PDF generado para el alumno.



YourCompany
250 Executive Park Blvd, Suite 3400
94134 San Francisco
California
España

INFORME DE SEGUIMIENTO ACADÉMICO

Expediente Histórico Consolidado

Emitido el: 23/04/2026 14:40

Alumno: Alumno Cinco
Email: alu5@ejemplo.com
Código: ALU005

Centro Docente: IES Triana
Tutor Asignado: Rafa Serna
Contacto Centro: ies.triana@juntadeandalucia.es

0.0
HORAS PREVISTAS

3.0
HORAS REALES

0
ASISTENCIAS

1
TAREAS

Concepto de Control	Valor Consolidado
Total acumulado de asistencias registradas	0 registros
Horas totales de ejecución (Tareas aprobadas)	3,0 h
Volumen total de entregas realizadas	1 tareas

Firma del Alumno

Sello del Centro / Tutor

Notas sobre Seguridad y Código

Durante el desarrollo, uno de los puntos más críticos fue la configuración de los archivos `ir.model.access.csv`. Sin una definición exacta de estos permisos, los módulos propios no eran visibles para el usuario.

5.5 Frontend y Experiencia de Usuario (React)

Arquitectura de Aplicación de Página Única (SPA)

El sistema se ha diseñado bajo el modelo de Single Page Application (SPA), lo que permite una navegación fluida sin recargas de página completas, mejorando la experiencia de usuario y la velocidad de respuesta. Toda la lógica de rutas y el control de vistas se gestionan en el lado del cliente, asegurando que la aplicación se comporte de manera ágil y moderna.

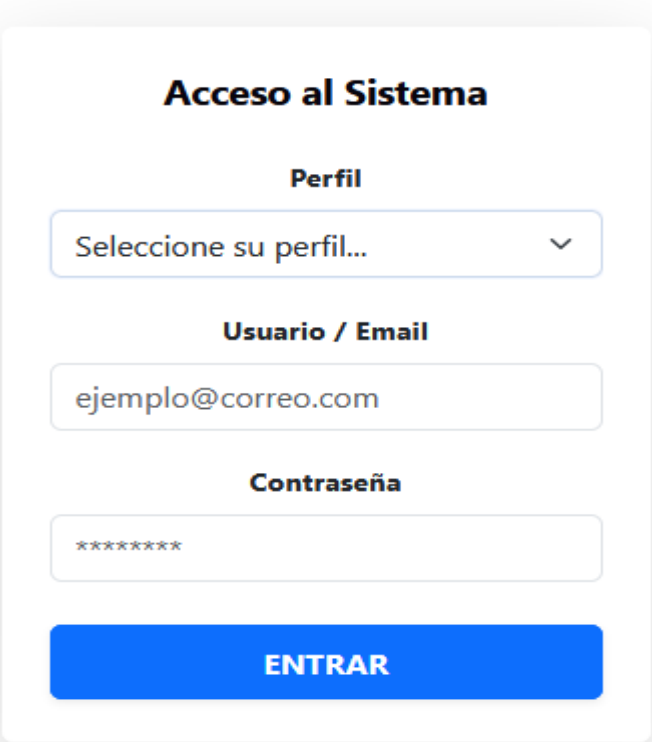
Gestión de Estado Global con Context API

Para el manejo de los datos compartidos en toda la aplicación, se ha implementado React Context API. Esta herramienta permite centralizar la información del usuario autenticado y las preferencias de la sesión, evitando el paso manual de datos entre componentes y garantizando que el estado sea consistente en cualquier parte del sistema.

Control de Acceso y Dashboards Independientes

La seguridad comienza en la vista de login, que actúa como puerta de entrada al sistema. Tras la validación de credenciales, el sistema identifica el rol del usuario y lo redirige a un Dashboard o vista independiente y personalizada. Cada perfil (profesor, alumno o empresa) cuenta con una interfaz exclusiva que muestra únicamente las herramientas y datos pertinentes a sus funciones.

Figura 16. Control de Acceso de la app



El formulario de acceso al sistema, titulado "Acceso al Sistema", está diseñado para permitir a los usuarios iniciar sesión. Incluye un selector de perfil, campos de entrada para el usuario y la contraseña, y un botón de entrada.

Acceso al Sistema

Perfil

Seleccione su perfil... ▼

Usuario / Email

ejemplo@correo.com

Contraseña

ENTRAR

Navegación Dinámica y Adaptabilidad (Responsividad)

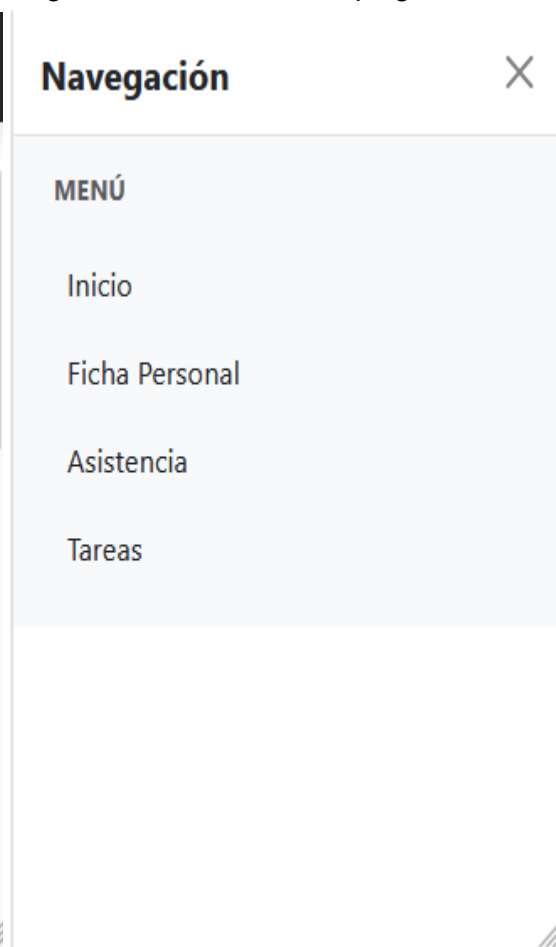
La interfaz utiliza una estructura compuesta por una barra de navegación superior (Navbar) y un menú lateral (Sidebar). Para optimizar el espacio en pantallas pequeñas, se ha implementado un sistema de Offcanvas:

- En dispositivos móviles, el Sidebar se oculta automáticamente y se activa mediante un icono de menú en la Navbar, desplegándose de forma lateral sobre el contenido.
- Este mismo componente Offcanvas se utiliza en la vista de tareas del profesor tutor para mostrar detalles específicos o formularios de edición sin perder el contexto de la lista principal.

Figura 17. Sidebar integrado en el Navbar



Figura 18. OffCanvas desplegado.



Componentes Reutilizables y Eficiencia

Se ha aplicado una estrategia de desarrollo basada en componentes reutilizables para mantener la coherencia visual y facilitar el mantenimiento:

- **AppTable:** Una tabla maestra configurada para mostrar cualquier tipo de dato con acciones integradas.
- **AppForm y AppModal:** Estructuras estandarizadas para la entrada de datos y ventanas emergentes.

Flujo Operativo en la Vista de Alumno

Se ha optimizado la interacción eliminando pasos innecesarios. En la gestión de tareas, el alumno ya no visualiza modales intermedios; al seleccionar un elemento de la lista, la información se carga dinámicamente en el panel superior, permitiendo una edición inmediata y fluida de sus actividades.

Gestión de Visualización: Paginación en Cliente (AppTable)

Para mejorar la experiencia de usuario y evitar interfaces saturadas de información, se ha desarrollado un componente reutilizable denominado AppTable. Este componente implementa una lógica de **paginación en el lado del cliente**, lo que permite una navegación instantánea entre registros sin tiempos de espera adicionales.

Detalles de la implementación técnica

- **Gestión de Memoria:** El sistema realiza una única petición al Backend para obtener el conjunto completo de datos, almacenándose en el estado de React.
- **Lógica de Segmentación:** Mediante el uso de hooks (useState) y métodos de array como slice(), el componente calcula dinámicamente los índices de los elementos que deben mostrarse en función de la página actual y el límite de elementos por página (itemsPerPage = 5).
- **Interactividad:** El cálculo de totalPages permite generar una interfaz de navegación reactiva que se adapta automáticamente al volumen de datos recibido desde el Backend.

Ventaja técnica: Esta aproximación es ideal para los volúmenes de datos actuales del proyecto (alumnos, empresas y tareas por centro), ya que reduce la carga del servidor al evitar peticiones constantes y ofrece una respuesta inmediata al usuario al cambiar de página.

6. Despliegue y Pruebas

Aunque el desarrollo de los módulos se realizó inicialmente en entornos locales individuales, la fase final del proyecto se centró en la **unificación mediante Docker**. Esta etapa de pruebas fue crítica para garantizar que la solución es portable y que todas las dependencias (bases de datos, ERP y modelos de IA) interactúan correctamente en un entorno aislado.

Pruebas de Orquestación Final

- **Transición a Contenedores:** Se validó que el código desarrollado en local no presentaba discrepancias al ejecutarse sobre imágenes de Docker, ajustando las variables de entorno para la comunicación interna.
- **Persistencia de Datos:** Se comprobó que, al utilizar volúmenes gestionados, los datos de MySQL y PostgreSQL (Odoo) sobreviven a los reinicios del servicio docker-compose, eliminando el riesgo de pérdida de información.
- **Resolución de Nombres (DNS Interno):** Se verificó que el Backend de Spring Boot apunta correctamente al host api-ml en lugar de localhost, haciendo uso de la red interna de Docker para una comunicación más segura y eficiente.

El proceso de validación tiene como objetivo asegurar que el ecosistema **FTC Gestión** cumple con los requisitos funcionales y técnicos definidos. Para este proyecto, se ha optado por un enfoque de **pruebas manuales de caja negra**, verificando el comportamiento del sistema desde la perspectiva del usuario y la integridad de las respuestas de la API.

Nota sobre la metodología: Se reconoce que, en un entorno de producción ideal, es imperativo implementar pruebas automatizadas (Unitarias con JUnit, de Integración y E2E). Debido a las restricciones temporales del proyecto, estas se han pospuesto para futuras iteraciones, priorizando en esta fase la validación del despliegue y la lógica de negocio crítica.

6.1 Puesta en marcha del sistema y despliegue

Procedimiento de despliegue con Docker Compose

Para garantizar una ejecución exitosa de FTC Gestión, es imperativo cumplir con una serie de requisitos técnicos antes de iniciar el proceso de despliegue:

- **Estado de Docker:** El motor de Docker (Docker Desktop en Windows/Mac o el demonio de Docker en Linux) debe estar instalado y en estado de ejecución (Running).
- **Disponibilidad de Puertos:** El sistema anfitrión debe tener libres los siguientes puertos, necesarios para la exposición de los microservicios: 5173 (React/Vite), 8088 (Spring Boot), 3308 (MySQL), 8000 (FastAPI), 8069 (Odoo) y 5432 (PostgreSQL para Odoo).

Una vez verificados los requisitos, el despliegue se realiza siguiendo estos pasos secuenciales:

1. **Descarga del Proyecto:** Se debe obtener el código fuente mediante la clonación del repositorio oficial de GitHub utilizando el comando `git clone https://github.com/tu-usuario/tu-repositorio.git` (<https://github.com/tu-usuario/tu-repositorio.git>) o descargando el archivo comprimido desde la misma URL.

2. **Localización del Orquestador:** Se abre una terminal y se accede al directorio raíz del proyecto, lugar donde se encuentra el archivo docker-compose.yml que centraliza toda la configuración.
3. **Levantamiento de Servicios:** Se ejecuta el comando de construcción y arranque: `docker-compose up --build`. Este proceso automatiza la descarga de imágenes oficiales, la compilación de los binarios de Java, la compilación del código React y la creación de la red virtual red-global-dam.

Al finalizar la ejecución del comando, todos los contenedores pasarán a estado operativo, permitiendo el acceso a las diferentes interfaces a través de sus respectivos puertos en el host local.

Figura 19. Vista de terminal una vez ejecutado el docker-compose up

```
PS C:\Users\davir\GSPFDAM\2DAM\proyecto_dam\proyecto-2dam-sistema-registro-practicas-formativas> docker-compose up -d
[+] Running 6/6
✔Container api-ml-python      Started           0.4s
✔Container db-postgres-odoo   Started           0.3s
✔Container mysql_dam_project  Started           0.5s
✔Container odoo_sge           Started           0.3s
✔Container api-spring-core    Started           0.3s
✔Container react-frontend-dam Started           0.4s
PS C:\Users\davir\GSPFDAM\2DAM\proyecto_dam\proyecto-2dam-sistema-registro-practicas-formativas> |
```

Configuración inicial e instalación de módulos en el ERP

Inicialización de la base de datos

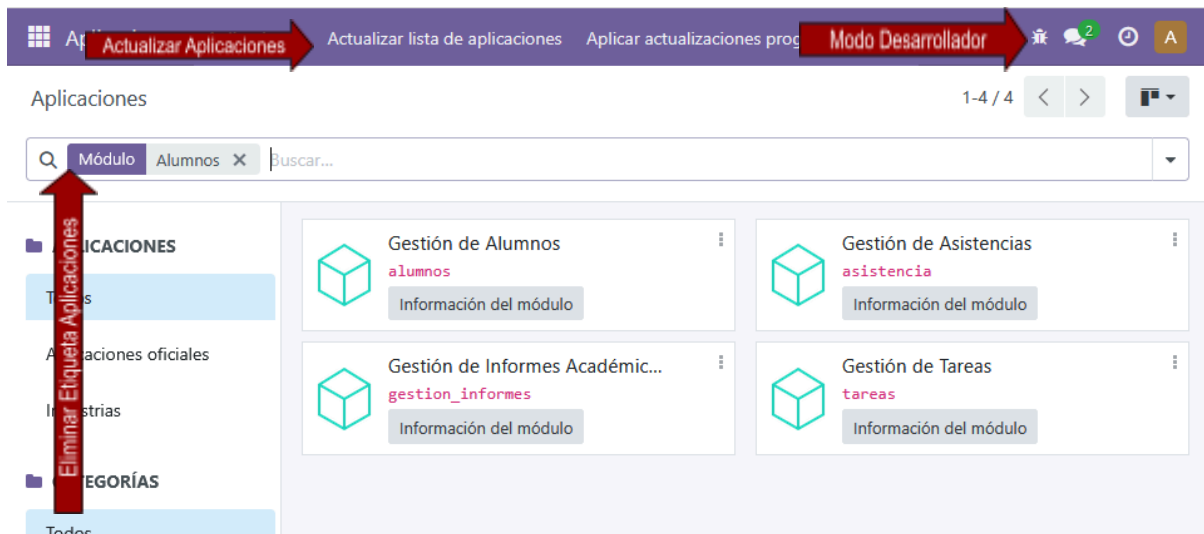
Al acceder por primera vez a la interfaz de Odoo (puerto 8069), se requiere la configuración del repositorio de datos. Este proceso implica la generación de una Master Password para la gestión administrativa y la creación de una base de datos específica (ej. odoo_practicas). Es fundamental no marcar la casilla de datos de demostración para garantizar que el repositorio permanezca vacío y listo para una sincronización limpia con el Backend de Spring Boot.

Ciclo de actualización y visibilidad de módulos

Debido al funcionamiento interno del ERP, la visibilidad de los addons personalizados vinculados mediante el volumen de Docker requiere un procedimiento técnico cíclico:

- Activación del modo desarrollador desde el panel de ajustes para habilitar las opciones de administración técnica.
- Ejecución de la actualización de la lista de aplicaciones para que el sistema escanee la carpeta de addons en busca de nuevos módulos.
- Gestión de filtros en el menú de Aplicaciones, eliminando el filtro por defecto para permitir la localización de los módulos de gestión propios.

Figura 20. Pantalla de carga módulos desarrollados.



Secuencia de instalación por dependencias

Para respetar la integridad del modelo de datos y las relaciones entre entidades, se debe seguir un orden estricto de instalación manual:

1. Infraestructura base: Instalación de los módulos de centro_docente y personal_docente.
2. Gestión del alumnado: Instalación secuencial de los módulos de alumno, asistencia, tareas y gestión_informes.
3. Capa de integración: Instalación final del componente api_connector, que habilita los endpoints necesarios para la comunicación con el resto del ecosistema.

6.2 Pruebas de integración y Validación APIs

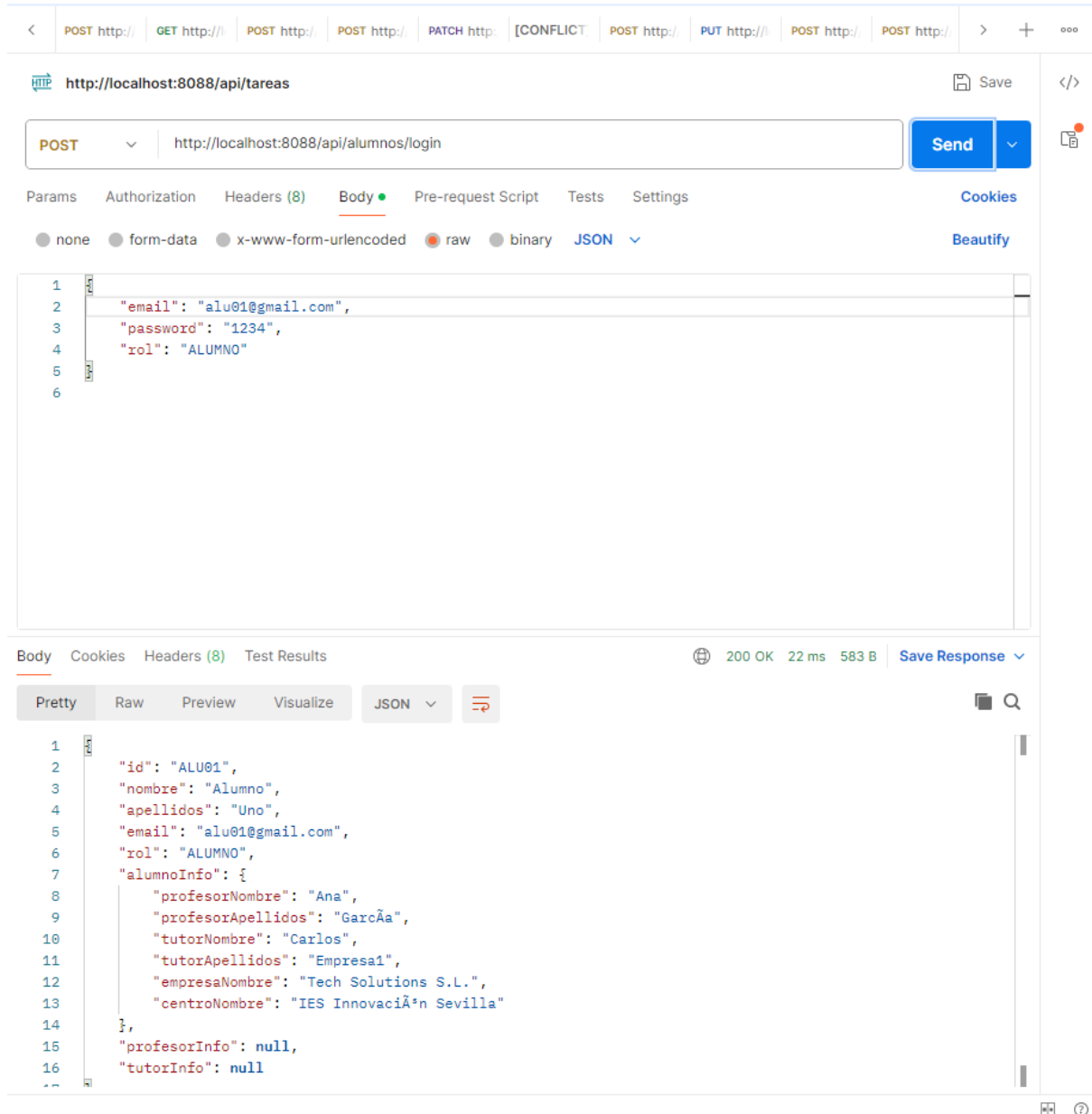
Validación del Backend con Postman

Una vez que los contenedores están operativos, se utiliza Postman para realizar pruebas de integración que aseguren la correcta comunicación entre el Backend (Spring Boot), la base de datos MySQL y el ERP Odoo. Estas pruebas validan que la lógica de negocio es sólida y que los datos fluyen sin errores:

- **Gestión de Sesión y Roles:** Se realizan peticiones POST al endpoint de autenticación para verificar que el sistema devuelve el token de acceso y identifica correctamente si el usuario es Alumno, Profesor o Gestor.
- **Operaciones CRUD de Alumnos y Tareas:** Se ejecutan peticiones GET para obtener listados y POST para crear nuevos registros. Se comprueba que el Backend procesa la información y la persiste correctamente en la base de datos MySQL.

- **Sincronización con Odoo:** Se testea el servicio de integración mediante peticiones que fuerzan el envío de datos al ERP, validando que el conector de Odoo recibe y registra la información sin discrepancias.

Figura 21. Método post aplicado al login en Postman



Pruebas de Microservicio de IA con Swagger

Para el módulo de Inteligencia Artificial desarrollado en Python, se utiliza la documentación interactiva de Swagger UI disponible en el puerto 8000. Esta herramienta permite probar de forma aislada la capacidad del modelo para analizar las tareas enviadas:

- **Análisis de Texto:** Se envían descripciones de tareas reales para comprobar que la IA devuelve un análisis de sentimiento y coherencia coherente.
- **Validación de Formatos:** Se asegura que el microservicio responde correctamente a los esquemas JSON esperados por el Backend de Spring Boot, garantizando una integración fluida.

Figura 22. Método Post aplicado a la predicción de horas en Swagger

The screenshot displays the Swagger UI interface for testing a microservice. At the top, there are 'Execute' and 'Clear' buttons. Below this, the 'Responses' section is active, showing the details of a successful POST request.

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "descripcion_completa": "Frontend Refactorización Baja Limpieza de estilos repetidos en el archivo CSS y creación de variables g
  }'
```

Request URL

```
http://localhost:8000/predict
```

Server response

Code	Details
200	Response body <pre>{ "horas_estimadas": 5.75 }</pre> Response headers <pre>content-length: 24 content-type: application/json date: Sun, 03 May 2026 15:25:54 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Resultados de las pruebas de integración

Las capturas anteriores demuestran la plena operatividad del ecosistema en el entorno Docker. La respuesta exitosa del servidor de autenticación confirma la integridad de la base de datos MySQL, mientras que las pruebas sobre el microservicio de Python validan la disponibilidad de las herramientas de inteligencia artificial para el análisis de tareas en tiempo real.

6.3 Validación de la Interfaz y Flujos de Usuario (Frontend)

Una vez validados los servicios de red y las APIs, se procede a verificar la capa de presentación desarrollada en React. El objetivo es confirmar que la interfaz consume correctamente los microservicios y respeta las reglas de negocio establecidas para cada perfil.

Control de Acceso y Seguridad de Rutas

Se han realizado pruebas de estrés sobre el sistema de autenticación, validando que el motor de seguridad bloquea el acceso en los siguientes escenarios:

- Introducción de credenciales erróneas (email/password).
- Discordancia entre el email y el rol seleccionado en el formulario de acceso.
- Intento de acceso mediante navegación por URL a rutas protegidas (ej. un alumno intentando entrar en /admin).

Figura 23. Error de login.

The image shows a web form titled "Acceso al Sistema". At the top, there is a red error message box that reads: "Error de autenticación. Verifique sus credenciales y el perfil seleccionado." Below this, the form has three main sections: "Perfil" with a dropdown menu currently showing "Profesor Gestor"; "Usuario / Email" with a text input field containing "alu01@gmail.com"; and "Contraseña" with a password input field represented by four dots. At the bottom of the form is a large blue button labeled "ENTRAR".

Ciclo de Vida de las Entidades y Roles

Se ha verificado la operatividad de las funciones principales para cada actor del sistema:

- **Gestor/Tutor:** Validación de los formularios de creación de entidades (Alumnos, Empresas y Centros), comprobando que los datos se envían al Backend y se refrescan en la UI sin necesidad de recarga manual.

Figura 24. Modal para añadir un alumno al sistema

FCT Gestión

MENÚ

- Inicio
- Gestión de Alumnos
- Gestión de Profesores
- Gestión de Empresas
- Gestión de Tutores de Empresa
- Gestión de Centros

Nuevo Alumno

Código (ID): ALU200

Horas Totales: [Empty]

Nombre: David

Apellidos: Romero

Email: david@prueba.es

Profesor Tutor: Ana GarcÃa

Empresa Asignada: Tech Solutions S.L.

Tutor de Empresa: Carlos Empresa1

Guardar Alumno

Cancelar

Matricular Alumno

Tratar por Empresa: [Dropdown]

Acciones

Profesor	Tutor	Acciones
GarcÃa		[Icons]
GarcÃa		[Icons]
GarcÃa		[Icons]
Ãrez		[Icons]
Ãrez		[Icons]

Figura 25. Tabla con el alumno guardado.

FCT Gestión

MENÚ

Inicio

Gestión de Alumnos

Gestión de Profesores

Gestión de Empresas

Gestión de Tutores de Empresa

Gestión de Centros

Gestión de Alumnos

Centro: IES Innovación Sevilla

Buscar por nombre o ID

Filtrar por Empresa

Todas las empresas

ID	Alumno	Empresa	Tutor Empresa	Profesor Tutor	Acciones
ALU200	David Romero	Tech Solutions S.L.	Carlos Empresa1	Ana García	

«

<

1

2

3

4

5

>

»

- **Alumno:** Registro de asistencias diarias y gestión de tareas. Se ha comprobado que el flujo de envío de tareas integra de forma transparente la respuesta de la API de IA, mostrando al alumno el feedback generado sobre su actividad.

Figura 26. Dashboard de Asistencias para Alumno

FCT Gestión

MENÚ

- Inicio
- Ficha Personal
- Asistencia
- Tareas

Gestión de Asistencia

Domingo, 3 De Mayo De 2026

Alumno: ALU01

ESTADO ACTUAL

En Jornada

HORA SALIDA

18:19

OBSERVACIONES

¿Qué has hecho hoy?

Cerrar Jornada

Historial de Jornadas

dd / mm / aaaa

Buscar

Fecha	Entrada	Salida	Horas	Observaciones	Acciones
2026-05-03	18:19:00	-	-	-	
2026-05-02	13:14:00	20:06:00	6.87 h	sin incidencias	
2026-05-01	10:36:00	16:17:00	5.68 h	sin incidentes	
2026-03-19	09:00:00	14:00:00	5.00 h	Corrección de errores en los estilos CSS.	
2026-03-18	08:15:00	14:15:00	6.00 h	Maquetación básica del Front-end en Angular.	

<<

<

1

2

>

>>

- **Interactividad y Estado:** Uso de la Context API para mantener la persistencia de la sesión del usuario durante la navegación y respuesta de los componentes ante acciones del usuario (notificaciones de éxito/error).

Página - 54

7. Conclusiones y Valoración

Tras la finalización del desarrollo y las pruebas del sistema **FCT Gestión**, se ha procedido a evaluar el grado de cumplimiento de los objetivos planteados al inicio de esta memoria. El resultado global es satisfactorio, habiendo superado incluso las expectativas iniciales mediante la inclusión de módulos no previstos originalmente.

7.1 Consecución de objetivos

Objetivos Específicos Alcanzados

- **Gestión Centralizada:** Se ha logrado unificar en una sola plataforma web (React) el acceso para los tres roles principales (alumno, tutor docente y tutor laboral), permitiendo una visibilidad compartida de la actividad del alumno en tiempo real.
- **Registro de Asistencia:** El sistema implementa un control de entrada y salida funcional, que además informa dinámicamente del estado del alumno ("En el centro de trabajo") mientras se encuentra dentro de su jornada laboral.
- **Gestión y Validación de Tareas:** El flujo de trabajo se ha completado con éxito. El tutor laboral asigna tareas y el tutor docente dispone de una herramienta de supervisión directa mediante un sistema de **validación por botón** en su panel de control.
- **Sincronización con ERP (Odoo):** Se ha cumplido el objetivo de crear un repositorio espejo. El sistema replica automáticamente todas las tablas operativas (alumnos, tareas, asistencias, etc.) en Odoo, exceptuando las entidades externas (empresa y tutor laboral), garantizando la persistencia y disponibilidad de los datos académicos en un entorno empresarial estándar.

Superación de Expectativas: El Módulo de IA

Aunque no formaba parte del alcance inicial definido en la introducción, se integró con éxito un servicio de **Machine Learning mediante FastAPI**.

- Este módulo permite realizar una regresión para estimar el tiempo de ejecución de las tareas basándose en datos históricos.
- **Madurez del modelo:** Se ha comprobado que, para garantizar la fiabilidad, es imprescindible el uso de selectores obligatorios que normalicen la entrada de datos.
- **Especialización:** Como hito técnico, se ha acotado el modelo específicamente para el ciclo de **DAM**, optimizando su precisión para este perfil profesional, aunque esto defina una limitación de uso para otros ciclos formativos en la versión actual.

Valoración de la Coordinación

A pesar de la ausencia de un chat interno (identificado como una futura mejora), la coordinación entre actores se ha visto drásticamente mejorada. El hecho de que el tutor docente pueda validar tareas desde la misma plataforma donde el alumno registra su progreso elimina la dispersión de herramientas y la pérdida de información que motivó el inicio de este proyecto.

7.2 Desafíos Transversales y Gestión de Riesgos

Más allá de las incidencias técnicas puntuales resueltas durante la codificación, el desarrollo de este ecosistema de microservicios planteó una serie de desafíos estructurales que requieren decisiones de diseño críticas:

- **Heterogeneidad Tecnológica:** El mayor reto estratégico fue la orquestación de cuatro entornos distintos (**Java/Spring, Python/FastAPI, JS/React y Odoo**). Mantener la integridad y la consistencia de los datos entre el flujo operativo y el "repositorio espejo" en Odoo requirió una lógica de sincronización más robusta de la prevista inicialmente, especialmente para asegurar que los estados de las tareas se reflejaran correctamente en todos los sistemas.
- **Normalización de Datos para Machine Learning:** La regresión de la IA resultó ser altamente sensible a la calidad de la entrada. El desafío consistió en equilibrar la experiencia de usuario (libertad para describir tareas) con la rigidez técnica necesaria. La solución de implementar **selectores obligatorios** fue determinante para evitar el ruido en los datos y asegurar que el modelo mantuviera un nivel de precisión útil para el entorno de DAM.
- **Curva de Aprendizaje y Adaptación de Odoo:** La integración del ERP supuso un reto de adaptación conceptual. Al tener una estructura interna y un ORM (OpenObject) que difieren significativamente del desarrollo a medida en Spring Boot, fue necesario realizar ajustes de compatibilidad en el diseño de la base de datos para que la comunicación mediante servicios externos fuera fluida y no penalizara el rendimiento.

7.3 Análisis de coste

Para evaluar la viabilidad económica del proyecto, se ha realizado una estimación de los costes dividida en recursos humanos e infraestructura técnica.

Costes de Personal (Recursos Humanos)

Se considera el rol de un **Analista Programador Junior** para el desarrollo completo del sistema. La dedicación estimada para la realización de este proyecto, incluyendo análisis, diseño, codificación y pruebas, es de aproximadamente **250 horas**.

Tabla Coste Personal			
Concepto	Horas	Precio/ Hora	Total
Análisis y Diseño: Arquitectura de microservicios y UML.	40	20 €	800 €
Desarrollo (Frontend + Backend + Modulos Odoo): Codificación de la lógica.	130	20 €	2.600 €

Integración e IA: Entrenamiento del modelo y conexión con Odoo.	40	20 €	800 €
Plan de Pruebas (QA): Validación de flujos, Docker y testeo.	22	20 €	500 €
Documentación Técnica: Elaboración de la memoria y anexos.	23	20 €	500 €
Total Personal	260		5.200 €

Costes de Software e Infraestructura

El uso de tecnologías *Open Source* ha permitido reducir los costes de licencias a cero. Para la puesta en marcha, se estima el coste de un servidor virtual (VPS) capaz de soportar la arquitectura de contenedores Docker.

Tabla Costes Software		
Concepto	Licencia	Coste estimado (Anual)
Entorno de ejecución (Docker, Java, Python, JS)	Open Source	0 €
Base de datos (PostgreSQL)	Open Source	0 €
ERP Odoo (Version Community)	LGPL	0 €
Servidor Cloud (Ej: AWS / DigitalOcean - 4GB RAM)	-	480 €
Total Infraestructura		480 €

Resumen de Inversión Total

Sumando ambos conceptos, el coste total para el desarrollo y el primer año de puesta en marcha del sistema sería de **5.480 €**. Cabe destacar que, al ser un software a medida, no existen cuotas de mantenimiento por usuario, lo que lo hace altamente rentable frente a soluciones comerciales propietarias.

7.4 Líneas de trabajo futuro

El proyecto **FCT Gestión** se ha diseñado sobre una arquitectura de microservicios escalable que permite la implementación de nuevas funcionalidades sin comprometer la estabilidad del sistema actual. Se proponen las siguientes líneas de mejora:

Interfaz y Experiencia de Usuario (UX/UI)

- **Pulido de Interfaz:** Realizar un ajuste general del diseño, corrigiendo comportamientos de los elementos interactivos en resoluciones *responsive* y fijando el tamaño del *sidebar* para garantizar la consistencia visual en toda la aplicación.
- **Pantalla de Primer Uso:** Implementar un asistente de configuración inicial (*Onboarding*) destinado al profesor gestor para facilitar la carga masiva de datos y la configuración inicial del centro.

Seguridad y Arquitectura

- **Robustez en el Acceso:** Sustituir el sistema de autenticación actual por uno basado en **JSON Web Tokens (JWT)** para mejorar la seguridad en la comunicación entre el Frontend y los microservicios.
- **Comunicación Bidireccional con Odoo:** Evolucionar la API de Spring Boot para que sea capaz de iniciar peticiones hacia Odoo (actualmente es Odoo quien consume de Spring). Esto permitiría validar y obtener datos de empresas y tutores directamente desde el ERP de forma proactiva.

Funcionalidades de Gestión y Comunicación

- **Módulo de Mensajería y Notificación:** Implementar un chat interno para centralizar la comunicación entre alumno, tutor docente y tutor laboral. Además de un sistema de avisos según cambia el estado de la tarea.
- **Sistema de Evaluación:** Añadir un apartado de anotaciones y calificaciones para que el profesor tutor pueda dejar un registro cualitativo del progreso del alumno más allá de la validación de tareas.
- **Integración Total de Módulos:** Acoplar de forma orgánica los módulos nativos de Odoo (gestión de empresas y trabajadores) con el flujo de gestión de prácticas personalizado.

Escalabilidad e Inteligencia Artificial

- **Entrenamiento Multiciclo:** Ampliar el dataset del modelo de Machine Learning para incluir datos de otros ciclos formativos (Sanidad, Mecánica, etc.), permitiendo que la estimación de tiempos de tareas sea precisa para cualquier familia profesional.
- **Multi-Instituto:** Adaptar la base de datos y la lógica de negocio para que la plataforma pueda ser utilizada por múltiples centros educativos de forma aislada (*Multi-tenancy*).

Despliegue y Calidad

- **Salto a la Nube:** Realizar el despliegue definitivo en un entorno Cloud (AWS o Azure) e incluir en la documentación técnica un manual detallado sobre la configuración de infraestructura en la nube.
- **Automatización de Pruebas:** Implementar una batería completa de **Test Unitarios y de Integración** automáticos para asegurar que las nuevas actualizaciones no generen regresiones en el sistema.

7.5 Conclusión final

Relación con los Módulos del Ciclo

El desarrollo de **FCT Gestión** ha permitido integrar de forma práctica los conocimientos adquiridos en el segundo curso del ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM), destacando especialmente la inclusión de formación específica del centro:

- **Acceso a Datos:** Implementación de la persistencia mediante JPA/Hibernate en Spring Boot para la gestión de la base de datos MySQL.
- **Desarrollo de Interfaces:** Creación de la SPA (Single Page Application) con React, gestionando la comunicación con el backend y la experiencia de usuario.
- **Programación de Servicios y Procesos:** Gestión de la comunicación entre microservicios y la orquestación de todo el ecosistema mediante contenedores Docker.
- **Sistemas de Gestión Empresarial:** Configuración y desarrollo del "repositorio espejo" en Odoo para centralizar la información administrativa.
- **Módulo de Inteligencia Artificial (Optativa del Centro):** Desarrollo del servicio de regresión en FastAPI para la estimación de tiempos de tareas, aplicando técnicas de normalización y manejo de datos para optimizar la precisión del modelo en el ámbito de DAM.

Reflexión Personal y Aprendizaje

El desarrollo de este proyecto ha supuesto un viaje de aprendizaje intenso y, en ocasiones, complejo. Durante la **fase de análisis**, la necesidad de reestructurar el diseño en varias ocasiones me ha enseñado que esta etapa, lejos de ser un trámite, es el pilar fundamental del software. Desde la inexperiencia, el análisis puede parecer una tarea secundaria, pero la realidad demuestra que consume tanto o más tiempo que la propia codificación.

En cuanto a la **fase de implementación**, el desarrollo ha sido una "pelea" constante con el código para lograr la estabilidad deseada. Sin embargo, ver hoy el sistema finalizado, con una orquestación en Docker que conecta con éxito múltiples entornos, representa el cumplimiento de un objetivo personal: pasar de los bucles de pirámides en C++ del primer trimestre de 1º de DAM a una arquitectura profesional y distribuida.

En conclusión, a pesar de las severas reestructuraciones —especialmente para integrar los módulos de Machine Learning y Odoo— y los constantes cambios durante el proceso, el resultado es un **MVP (Producto Mínimo Viable) óptimo**. Es un sistema completo que no solo cumple con los requisitos intermodulares, sino que sienta una base sólida para futuras evoluciones en el ámbito de la gestión educativa.

8. Bibliografía

Para el desarrollo del proyecto se han consultado las siguientes fuentes de documentación oficial y recursos técnicos:

8.1 Documentación oficial de tecnologías

- **Spring Framework:** Documentación oficial de Spring Boot y Spring Data JPA. Disponible en: <https://spring.io/projects/spring-boot>
- **React:** Documentación de referencia para el desarrollo de interfaces y gestión de estado. Disponible en: <https://react.dev/>
- **FastAPI:** Manual de referencia para la creación de APIs en Python y manejo de tipado. Disponible en: <https://fastapi.tiangolo.com/>
- **Odoo:** Guía de desarrollo para la versión Community y servicios web. Disponible en: <https://www.odoo.com/documentation/17.0/>
- **Docker:** Guía de referencia para Docker Engine y Docker Compose. Disponible en: <https://docs.docker.com/>

8.2 Documentación teórica asignaturas

- **Sistemas de Gestión Empresarial (SGE):** Guías de configuración y apuntes teóricos sobre el ciclo de vida y arquitectura de Odoo proporcionados por el equipo docente del módulo.
- **Inteligencia Artificial (Optativa):** Material didáctico, casos prácticos y guías de implementación de modelos de regresión facilitados por el profesorado de la asignatura. Este material ha sido la base fundamental para el entrenamiento y ajuste del modelo de predicción de tiempos en DAM.

8.3. Herramientas de Asistencia y Productividad

- **Modelos de Lenguaje de Gran Escala (LLM):** Se ha hecho uso de asistentes de IA generativa como apoyo en la estructuración de la memoria técnica, optimización de bloques de código específicos y revisión sintáctica del documento final.
- **Stack Overflow:** Consulta de hilos de resolución de errores comunes en la integración de microservicios y gestión de dependencias en Maven y NPM.